Population-Based Training with Meta-Strategy Alignment for Zero-Shot Coordination in Hanabi

by

Thomas Kelly

This thesis has been submitted in partial fulfillment for the degree of Master of Science in Artificial Intelligence

in the
Faculty of Engineering and Science
Department of Computer Science

May 2026

Declaration of Authorship

This report, Population-Based Training with Meta-Strategy Alignment for Zero-Shot Coordination in Hanabi, is submitted in partial fulfillment of the requirements of Master of Science in Artificial Intelligence at Munster Technological University Cork. I, Thomas Kelly, declare that this thesis titled, Population-Based Training with Meta-Strategy Alignment for Zero-Shot Coordination in Hanabi and the work represents substantially the result of my own work except where explicitly indicated in the text. This report may be freely copied and distributed provided the source is explicitly acknowledged. I confirm that:

- This work was done wholly or mainly while in candidature Master of Science in Artificial Intelligence at Munster Technological University Cork.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at Munster Technological University Cork or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this project report is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed: _____

Date: _____

MUNSTER TECHNOLOGICAL UNIVERSITY CORK

Abstract

Faculty of Engineering and Science
Department of Computer Science

Master of Science

by Thomas Kelly

Cooperative Multi-Agent Reinforcement Learning (MARL) requires agents to coordinate under partial observability, yet standard self-play training produces brittle conventions that fail when independently trained agents are paired at deployment, a problem known as zero-shot coordination (ZSC). This thesis addresses ZSC on the Tiny Hanabi benchmark by adapting the Extensive-form Double Oracle (XDO) algorithm to the cooperative setting. A Recurrent Proximal Policy Optimisation (RPPO) oracle best-responds to a growing population of Behaviour Cloning partner profiles across training iterations, exposing each agent to diverse coordination conventions rather than a fixed self-play partner.

A central challenge identified in this work is meta-strategy divergence: as two jointly trained agents update their strategy distributions independently via multiplicative weights, role asymmetry causes the distributions to concentrate on incompatible conventions, resulting in poor cross-play performance. To address this, a novel meta-strategy alignment mechanism is introduced, controlled by a coupling parameter α that blends each agent's update signal with their partner's probe scores. The L1 divergence between the two agents' meta-strategies serves as a diagnostic metric, and the cross-play score and XP/SP ratio are used as primary zero-shot coordination outcome metrics.

Experimental results demonstrate that coupling ($\alpha = 0.5$) successfully bounds meta-strategy divergence and improves cross-play performance by 18% over uncoupled agents (2.982 vs 2.530). An auxiliary partner-prediction loss is introduced to encode convention information into the recurrent hidden state, and a lightweight test-time adaptation method, Convention-Conditioned Belief blending, is evaluated, achieving a further gain of 3.7% over the fixed best-response baseline without any gradient updates. Gradient-based fine-tuning methods are shown to be ineffective at this scale due to insufficient partner-prediction signal, highlighting the importance of training-time convention alignment over test-time adaptation.

Acknowledgements

I would like to thank Dr Hamdan Awan for his help.

Contents

List of Figures

List of Tables

Abbreviations

AI	**A**rtificial **I**ntelligence
BC	**B**ehaviour **C**loning
CB	**C**onvention-Conditioned **B**elief (blending)
CFR	**C**ounterfactual **R**egret minimisation
Dec-POMDP	**Dec**entralised **P**artially **O**bservable **M**arkov **D**ecision **P**rocess
FCP	**F**ictitious **C**o-**P**lay
GAE	**G**eneralised **A**dvantage **E**stimation
GPU	**G**raphics **P**rocessing **U**nit
GRU	**G**ated **R**ecurrent **U**nit
JIT	**J**ust-**I**n-**T**ime (compilation)
LL	**L**og-**L**ikelihood
MAML	**M**odel-**A**gnostic **M**eta-**L**earning
MARL	**M**ulti-**A**gent **R**einforcement **L**earning
MDP	**M**arkov **D**ecision **P**rocess
NE	**N**ash **E**quilibrium
ODCFR	**O**pponent-Modelling **D**eep **C**ounterfactual **R**egret minimisation
PPO	**P**roximal **P**olicy **O**ptimisation
RL	**R**einforcement **L**earning
RPPO	**R**ecurrent **P**roximal **P**olicy **O**ptimisation
SP	**S**elf-**P**lay
VRAM	**V**ideo **R**andom **A**ccess **M**emory
XDO	E**x**tensive-form **D**ouble **O**racle
XP	**C**ross-**P**lay
ZSC	**Z**ero-**S**hot **C**oordination

Chapter 1

Introduction

This chapter motivates the research problem, states the contributions, and outlines the structure of the document.

1.1 Motivation

Artificial Intelligence has achieved significant milestones in mastering complex strategic games. Early breakthroughs such as DeepMind's AlphaZero demonstrated that machines could surpass human champions in games like Chess, Shogi, and Go [1]. However, these successes were in perfect-information games, where all players have complete visibility of the game state. The next frontier involves imperfect-information games, where agents must reason about hidden information and coordinate based on incomplete observations. The cooperative card game Hanabi has emerged as a key benchmark for this challenge [2]. Hanabi presents a unique coordination problem: players can observe their teammates' cards but not their own, requiring them to infer information through communication and behaviour patterns. Success in Hanabi demands what researchers call a "theory of mind": the ability to model what others know and adapt one's strategy accordingly [3]. This makes it an ideal test-bed for studying multi-agent coordination under information asymmetry.

The core motivation for this thesis is to address the zero-shot coordination problem in cooperative imperfect-information games. By employing the Extensive-form Double Oracle (XDO) framework [4] with a Recurrent Proximal Policy Optimisation (RPPO)

oracle, agents are trained against a growing population of diverse partner strategies rather than a single self-play partner. A key challenge when applying population-based training to cooperative settings is that independently trained agents may develop incompatible conventions even when each performs well within its own training run. To address this, a meta-strategy alignment mechanism is introduced that steers both agents' strategy distributions toward compatible conventions during joint training, with the goal of improving zero-shot coordination at deployment.

Beyond training-time alignment, a further challenge arises at deployment: even when agents have been trained toward compatible conventions, an agent paired with an unknown partner must identify which convention the partner is using and adapt its behaviour accordingly. This work also investigates test-time adaptation strategies for this inference problem, introducing an auxiliary partner-prediction loss to encode convention information into the agent's recurrent hidden state during training, and evaluating a lightweight Convention-Conditioned Belief blending approach that requires no parameter updates at deployment.

This research direction is motivated by both theoretical and practical considerations. Theoretically, it explores whether population-based game-theoretic methods can be successfully adapted to cooperative MARL problems that have traditionally been the domain of pure reinforcement learning approaches [5]. Practically, it addresses a critical deployment challenge: creating multi-agent systems that can coordinate flexibly with partners they have never encountered during training, a requirement for real-world applications ranging from autonomous vehicle coordination to collaborative robotics.

1.2 Contribution

The central problem addressed by this research is the fragility of learned conventions in cooperative multi-agent systems. Existing Hanabi agents typically develop rigid coordination protocols through self-play, which perform well with familiar partners but fail when paired with agents employing different conventions. This lack of adaptability represents a fundamental limitation: agents assume that their partner will follow the same implicit rules learned during training, and even minor deviations cause coordination to break down entirely.

1.2.1 Research Questions

This thesis investigates four core questions:

1. Can a population-based training framework using XDO with a Recurrent PPO oracle produce agents capable of effective zero-shot coordination on the Tiny Hanabi benchmark?
2. How does meta-strategy alignment via the coupling parameter affect the L1 divergence between jointly trained agents, and what is its measurable effect on cross-play performance?
3. What is the relationship between within-run meta-strategy divergence and same-method cross-play performance, and can L1 divergence serve as a reliable predictor of zero-shot coordination capability?
4. Can test-time convention inference be improved by training agents with an auxiliary partner-prediction loss, and does Convention-Conditioned Belief blending provide a more effective coordination signal than gradient-based fine-tuning at deployment?

1.2.2 Research Objectives

The technical objectives of this work are:

- **Implementation of XDO with RPPO oracle:** Develop a recurrent actor-critic policy trained via Proximal Policy Optimisation that best-responds to a population mixture of partner strategies, operating within the Extensive-form Double Oracle outer training loop.
- **Population diversity via XDO:** Employ the Double Oracle framework to train agents against a growing population of Behaviour Cloning profiles rather than a single self-play partner, with the goal of improving generalisation to unseen coordination protocols.
- **Meta-strategy alignment:** Introduce and evaluate a coupling mechanism that steers both agents' multiplicative-weights meta-strategies toward compatible conventions during joint training, assessed via L1 divergence and cross-play evaluation.

- **Test-time adaptation:** Train a partner-prediction auxiliary head alongside the RPPO oracle to encode convention information in the GRU hidden state, and evaluate Convention-Conditioned Belief blending — a parameter-free logit interpolation method — against gradient-based fine-tuning methods as test-time coordination strategies.

1.2.3 Main Contribution

This research advances the state of the art in cooperative zero-shot coordination by combining population-based training through XDO with a novel meta-strategy alignment mechanism. Rather than relying on symmetry-based convention grounding, as in Other-Play [6], or a fixed diverse partner pool, as in Fictitious Co-Play [7], the proposed framework explicitly steers jointly trained agents toward compatible convention spaces during training via a coupling parameter that blends probe scores across agents. The primary diagnostic metric is L1 divergence between the two agents' meta-strategies; the primary outcome metric is the cross-play score — the standard zero-shot coordination evaluation from the literature — measuring whether independently seeded agents trained with the same method can coordinate effectively at deployment time.

Additionally, an auxiliary partner-action prediction loss is introduced during oracle training to encode convention information explicitly into the recurrent hidden state. At test time, a Convention-Conditioned Belief blending strategy is evaluated, which interpolates each agent's best-response logits with those of the inferred Behaviour Cloning profile without any gradient computation. Across 500 evaluation episodes, this achieves a further 3.7% gain over the fixed best-response baseline, while gradient-based fine-tuning and adapter methods are shown to be ineffective at this scale due to insufficient partner-prediction signal strength, highlighting the importance of training-time convention alignment over test-time parameter adaptation.

1.3 Structure of This Document

This thesis is organised into six chapters:

- **Chapter 2** reviews relevant literature in multi-agent reinforcement learning, game theory, and zero-shot coordination, covering the Hanabi benchmark, Counterfactual Regret Minimisation, and population-based training methods including PSRO and XDO, positioning this work within the context of cooperative ZSC research.

- **Chapter 3** presents the formal problem definition and system design, including the Dec-POMDP formulation for Tiny Hanabi, the XDO training loop structure, the RPPO oracle architecture, the meta-strategy alignment mechanism, and the auxiliary partner-prediction loss.

- **Chapter 4** describes the full implementation, covering the JAX/Flax framework [8, 9], the RPPOActorCritic network with partner prediction head, the XDORPPOHanabiSolverJax, Behaviour Cloning profile generation, meta-strategy alignment implementation, and the deviation from the original ODCFR design to RPPO.

- **Chapter 5** presents experimental results across three phases: within-run training diagnostics (L1 divergence, meta-strategy entropy, probe scores, BC accuracy); cross-play evaluation comparing uncoupled (α=0) and coupled (α=0.5) agents; and test-time adaptation evaluation comparing auxiliary loss convention encoding, Convention-Conditioned Belief blending across a β sweep, and gradient-based fine-tuning methods.

- **Chapter 6** discusses the findings, draws conclusions about the effectiveness of meta-strategy alignment and test-time belief blending for zero-shot coordination, and identifies directions for future work.

Chapter 2

Literature Review

This chapter surveys the literature relevant to cooperative multi-agent reinforcement learning, zero-shot coordination, and test-time adaptation, positioning this work within the existing body of research.

2.1 Overview of Multi-Agent Reinforcement Learning

Multi-agent reinforcement learning (MARL) has undergone substantial evolution over the past decade, progressing from tabular methods to deep neural architectures capable of handling high-dimensional state spaces. The field is typically categorised by the interaction structure between agents: fully cooperative, fully competitive, or mixed-motive environments [5]. In cooperative settings, exemplified by Hanabi, the fundamental challenge is the Decentralised Partially Observable Markov Decision Process (Dec-POMDP) [10]. Unlike single-agent reinforcement learning, where the environment dynamics remain stationary during training, MARL agents face non-stationarity: as one agent's policy evolves, it changes the effective environment observed by other agents, potentially invalidating previously learned strategies [11]. This "moving target" problem represents a core challenge that any multi-agent coordination framework must address.

2.2 Coordination Frameworks in Cooperative MARL

Researchers have developed several structural frameworks to address the Dec-POMDP coordination challenge, each with distinct trade-offs between simplicity, performance, and scalability. Table 2.1 summarises the primary approaches.

Independent Learning approaches, while conceptually simple, struggle with environmental non-stationarity as each agent's policy changes affect others' learning dynamics. Value factorisation methods such as QMIX [12] achieve strong empirical results by learning a joint value function that decomposes into agent-specific contributions, but require access to global state information during centralised training. Centralised critic approaches like MADDPG [13] use a global critic to stabilise training while maintaining decentralised execution, but at a significant computational cost. Game-theoretic methods, particularly those based on Counterfactual Regret Minimisation (CFR), provide convergence guarantees to Nash equilibrium and handle imperfect information naturally, making them particularly relevant to this research.

TABLE 2.1: Summary of MARL Frameworks

Framework	**Description**	**Key Algorithms**	**Pros/Cons**
Independent Learning	Each agent treats others as part of the environment	IQL, IDRQN	Simple; prone to non-stationarity.
Value Factorisation	Centralised training of a joint Q-value, decomposed into local values.	VDN, QMIX, QPLEX	Strong performance; requires "Global State" during training.
Centralised Critics	Policy gradients with a "Global Critic" to reduce variance.	MADDPG, COMA	Addresses credit assignment; high computational cost.
Game-Theoretic	Iterative calculation of Best Responses and Regret.	CFR, Deep CFR, XDO	Mathematically grounded; scales to imperfect information.

2.3 The Hanabi Benchmark and Zero-Shot Coordination

The Hanabi Challenge [2] was introduced to expose fundamental limitations of the self-play training paradigm. While self-play has achieved remarkable success in perfect-information games, most notably AlphaZero's superhuman performance in Chess, Shogi,

and Go [1], it produces fragile coordination strategies in imperfect-information cooperative settings. Agents trained through self-play develop what Bard et al. call "ad hoc conventions": arbitrary coordination protocols that achieve high performance with training partners but fail catastrophically when paired with agents using different conventions. High-performing Hanabi agents such as the Simplified Action Decoder (SAD) [14] demonstrate that strong within-run performance does not imply robustness to partner variation.

This fragility motivated research into Zero-Shot Coordination (ZSC), which evaluates an agent's ability to coordinate with partners it has never encountered during training. Hu et al. [6] introduced Other-Play (OP), which leverages environmental symmetries to encourage agents to learn conventions grounded in the game's structure rather than arbitrary training artefacts. This approach significantly improves cross-play performance by ensuring learned conventions generalise across different agent implementations.

Later work has explored alternative approaches to ZSC. Off-Belief Learning (OBL) [15] explicitly models the gap between an agent's actual beliefs and what it signals to partners, improving coordination under belief asymmetry. K-Level Reasoning [16] extends this by modelling hierarchical levels of strategic thinking: an agent considers not only what its partner will do, but also what its partner thinks the agent will do. These methods represent different points in the design space between learning fixed conventions through standard self-play and dynamically adapting to partner behaviour.

2.4 Counterfactual Regret Minimisation in Imperfect-Information Games

While reinforcement learning methods optimise expected cumulative reward through gradient-based policy updates, game-theoretic approaches focus on regret minimisation. Counterfactual Regret Minimisation (CFR) [17] has become the dominant algorithm for solving large imperfect-information games, achieving superhuman performance in poker variants through systems such as Libratus and Pluribus.

CFR operates by iteratively computing the regret for not having taken alternative actions at each decision point, where regret is measured counterfactually: considering

what would have happened had a different action been chosen while all other decisions remained fixed. By minimising cumulative regret, CFR converges to a Nash equilibrium strategy with provable guarantees [17]. However, classical CFR maintains tabular regret values for every information set, limiting its applicability to games with tractable state spaces.

Deep CFR [18] addresses this scalability limitation by replacing tabular regret storage with neural network function approximators. The algorithm maintains two networks: a regret network that predicts counterfactual regret values for each action, and a strategy network that learns the average strategy over training iterations. This enables CFR to scale to games with state spaces too large for tabular methods, though at the cost of losing exact convergence guarantees.

A critical limitation of standard CFR is the assumption of a stationary opponent. In cooperative settings where partners are also learning, this assumption is violated. Opponent Modelling Deep CFR (ODCFR) [19] addresses this by integrating an explicit opponent model that identifies partner strategy types and adapts counterfactual regret computation accordingly. Wang et al. demonstrate that this approach enables agents to exploit suboptimal partner behaviour while maintaining near-Nash equilibrium guarantees against competent opponents. While the oracle in this thesis uses a reinforcement learning approach rather than CFR directly, the game-theoretic framing of best-response computation and the population-based training structure of XDO are both rooted in this body of work.

2.5 Extensive-Form Double Oracle and Population-Based Training

The Double Oracle algorithm [20] iteratively builds a restricted game by maintaining a small set of strategies and expanding this population when new best responses are found, providing computational efficiency while maintaining convergence to Nash equilibrium.

Policy-Space Response Oracles (PSRO) [21] extends Double Oracle to multi-agent reinforcement learning by using neural network-based oracles to compute approximate best responses. PSRO maintains a population of policies and trains new agents against

mixtures of existing strategies, enabling population-based training in complex domains. However, PSRO operates on normal-form game representations, which suffer from exponential explosion in sequential games: each pure strategy must specify actions for all decision points, making the representation intractable for games like Hanabi.

Extensive-Form Double Oracle (XDO) [4] addresses this limitation by operating directly on extensive-form game trees. By representing strategies as policies over information sets rather than complete action sequences, XDO achieves exponential computational savings while maintaining PSRO's population-based training structure. XDO integrates naturally with extensive-form algorithms such as Deep CFR [18], which serves as the best-response oracle in the original formulation.

The significance for Zero-Shot Coordination lies in population diversity. By training against multiple strategies rather than a single self-play partner, XDO-trained agents develop robustness to strategic variation [4]. This thesis leverages XDO as the outer training loop, with a Recurrent PPO oracle providing the best-response computation at each iteration.

2.6 Population Diversity Methods for Zero-Shot Coordination

The recognition that self-play produces brittle conventions has motivated several methods that explicitly pursue behavioural diversity during training. Fictitious Co-Play (FCP) [7] constructs a diverse partner pool by saving agent checkpoints at multiple stages of training and exposing the final agent to this range of behaviours. By training against partners with varied skill levels and conventions, FCP significantly improves cross-play performance over self-play baselines. However, FCP does not provide an explicit mechanism for aligning the conventions adopted by independently trained agents: the diversity is in the training pool, not in a principled approach to convention agreement.

Maximum Entropy Population-Based Training (MEP) [22] extends population-based methods by explicitly maximising the entropy of the strategy population, using a diversity bonus to encourage agents to develop conventions that are meaningfully distinct

from one another. This improves coverage of the convention space during training, but similarly provides no guarantee that independently trained instances will converge on mutually compatible conventions at deployment.

TrajeDi [23] takes a complementary approach by maximising diversity at the trajectory level, training agents to follow episode trajectories that are statistically distinguishable from those of other population members. This encourages behavioural diversity throughout the episode rather than only at the level of strategy selection. Collectively, these methods establish that population diversity is beneficial for zero-shot coordination, but none explicitly address the meta-strategy alignment problem that arises when two independently trained agents are paired at test time.

2.7 Test-Time Adaptation for Zero-Shot Coordination

A complementary strand of research addresses the coordination problem from the deployment side: rather than modifying training to produce compatible conventions, these methods adapt agent behaviour online using observations of the partner during evaluation. This is particularly relevant when agents from different training runs must coordinate without any shared training signal.

Partner type inference is a natural framing for this problem. If an agent can identify which convention its partner is following from early observations, it can switch to the best-response strategy for that convention. Log-likelihood inference over a library of Behaviour Cloning profiles provides a tractable mechanism: the agent scores each profile by the likelihood that it would have produced the observed partner actions, and selects the best-response checkpoint associated with the highest-scoring profile. This approach requires no gradient computation at deployment and scales linearly with the size of the profile library.

Off-Belief Learning [15] demonstrates that explicitly tracking partner belief states can substantially improve coordination when partners hold different prior beliefs about the game state. This motivates approaches that condition agent behaviour on inferred partner beliefs rather than fixed training conventions. The recurrent architecture used in this thesis — a Gated Recurrent Unit (GRU) [24] accumulating game history in its hidden

state — provides a natural substrate for online convention inference, as the hidden state implicitly encodes information about partner behaviour patterns observed so far.

Convention-Conditioned Belief (CB) blending extends this idea by interpolating an agent's best-response action logits with the logits of the inferred Behaviour Cloning profile at each step. Rather than a hard switch to the inferred best response, the blend allows the agent to soften its action distribution toward the partner's expected convention without committing fully. This approach is closely related to the belief blending strategies studied in theory of mind research [3], where an agent modulates its behaviour based on a continuously updated model of partner intent. Unlike gradient-based fine-tuning methods such as adapter modules or last-layer updates, CB blending adds no trainable parameters and introduces no optimisation instability at test time, making it a lightweight and robust alternative for cooperative ZSC.

2.8 Research Gap

While existing work has made substantial progress on zero-shot coordination, two key gaps remain. First, population-based methods such as FCP [7] and XDO [4] train agents against diverse partner pools but provide no mechanism to ensure that jointly trained agents converge on *compatible* conventions. Agents may develop independently effective strategies that nonetheless fail to coordinate when paired at deployment. Methods like Other-Play [6] address convention alignment via environmental symmetry constraints, but require structural knowledge of the game and do not generalise across arbitrary partner populations or training paradigms.

Second, even when training-time alignment is achieved, no existing method in the cooperative ZSC literature directly addresses the test-time convention identification problem for agents trained via population-based methods. Off-Belief Learning [15] and K-Level Reasoning [16] approach coordination through belief modelling but are not designed to operate over a library of pre-trained best-response checkpoints. Gradient-based test-time adaptation methods from the broader machine learning literature have not been systematically evaluated against lightweight logit-blending alternatives in the cooperative ZSC setting.

This thesis addresses both gaps. The meta-strategy alignment mechanism introduced within the XDO training loop explicitly steers both agents' multiplicative-weights strategy distributions toward compatible conventions during joint training. The mechanism is evaluated empirically via L1 divergence between the agents' meta-strategies as a diagnostic, and cross-play score as the primary zero-shot coordination outcome metric. Additionally, Convention-Conditioned Belief blending is introduced and evaluated as a parameter-free test-time adaptation strategy, compared systematically against gradient-based fine-tuning and adapter methods.

Chapter 3

Design

This chapter formalises the problem addressed by this thesis, states the design objectives, specifies functional and non-functional requirements, and describes the high-level system design. The detailed implementation of each component is deferred to Chapter 4.

3.1 Problem Definition

3.1.1 Tiny Hanabi as a Dec-POMDP

The cooperative card game Hanabi and its tractable variant Tiny Hanabi are formalised as Decentralised Partially Observable Markov Decision Processes (Dec-POMDPs) [25]. A Dec-POMDP for n agents is defined by the tuple $(S, \mathbf{A}, T, R, \mathbf{\Omega}, O, \gamma)$, where:

- S is the set of environment states, encompassing the full deck composition, each player's hand, the discard pile, the number of remaining hint tokens, and the number of lives.
- $\mathbf{A} = A_1 \times A_2$ is the joint action space. Each agent's action set A_i comprises: playing a card from their hand, discarding a card, and providing a hint to the partner specifying either a colour or a rank value.
- $T : S \times \mathbf{A} \times S \to [0, 1]$ is the transition function governing how the game state evolves following a joint action.

- $R : S \times \mathbf{A} \to \mathbb{R}$ is the shared reward function. A reward of $+1$ is received when a card is successfully played in the correct position on the fireworks pile; a reward of -1 is incurred when a life is lost through an illegal play. All other actions yield zero reward. The team's cumulative score is the primary outcome metric.
- $\mathbf{\Omega} = \Omega_1 \times \Omega_2$ is the joint observation space. Agent i observes the partner's full hand, the discard pile, the remaining hint tokens, the number of lives, and all hints given during the current episode, but crucially *cannot* observe their own hand. This information asymmetry is the defining challenge of the game.
- $O : S \times \mathbf{A} \times \mathbf{\Omega} \to [0, 1]$ is the observation function encoding the conditional probability of each joint observation given the current state.
- $\gamma \in [0, 1]$ is the discount factor, set to 1 for the episodic Hanabi setting.

Tiny Hanabi is a reduced variant of the full game designed to make iterative research tractable while preserving the core coordination challenge. The version used in this thesis comprises two players, a deck of 8 cards drawn from 2 colours and 2 rank values, a hand size of 2 cards per player, and a maximum achievable score of 4. The reduced state space permits many XDO iterations within a single compute session while still exhibiting meaningful convention diversity, making it a standard benchmark for zero-shot coordination research [6].

3.1.2 Zero-Shot Coordination Problem Statement

Zero-Shot Coordination (ZSC) evaluates a training method's ability to produce agents that can coordinate with independently trained partners at deployment, without any joint fine-tuning. Formally, let $\mathcal{T}$ be a training procedure that, given a random seed ω, produces a pair of agents (A^ω, B^ω) trained together under that seed.

The within-run *self-play* (SP) score is the expected team score when agents from the same run play together:

$$\mathrm{SP} = \mathbb{E}_\omega \left[V(A^\omega, B^\omega) \right]$$

where $V(x, y)$ denotes the mean episode score when agent x acts as player 0 and agent y acts as player 1.

The *cross-play* (XP) score measures coordination between agents from *different* runs of the same training method. For two independent seeds $\omega_1 \neq \omega_2$, the symmetric XP score is:

$$\text{XP} = \frac{1}{2}\left[V(A^{\omega_1}, B^{\omega_2}) + V(A^{\omega_2}, B^{\omega_1})\right]$$

The XP/SP ratio is the primary ZSC evaluation metric used in this thesis [6]:

$$\text{ZSC ratio} = \frac{\text{XP}}{\text{SP}}$$

A ratio of 1.0 indicates that agents coordinate equally well with independently trained partners as with their own training partner. A ratio close to 0 indicates convention over-fitting: high within-run performance that collapses entirely in cross-play.

3.1.3 The Meta-Strategy Alignment Problem

In the XDO training loop, each agent $i \in \{A, B\}$ maintains a meta-strategy $\boldsymbol{w}_i^{(t)} \in \Delta^K$ over a population of K Behaviour Cloning (BC) profiles accumulated over t iterations, where Δ^K denotes the K-dimensional probability simplex. The meta-strategy is updated via multiplicative weights:

$$w_i^{(t+1)}[k] \propto w_i^{(t)}[k] \cdot \exp(s_i[k])$$

where $s_i[k]$ is the probe score of profile k evaluated against agent i's current oracle policy.

Because agents A and B occupy asymmetric roles in Tiny Hanabi (player 0 and player 1 have different observation and action structures), the same profile k will generally score differently under each agent's probe evaluation: $s_A[k] \neq s_B[k]$. Multiplicative weights amplifies these initial score differences over successive iterations, causing the two agents' meta-strategies to diverge. In the limit, each agent's weight distribution concentrates on a different subset of the profile pool, meaning agent A specialises against convention X while agent B specialises against convention Y. When paired across runs, the agents play incompatible conventions, causing cross-play scores to collapse.

This divergence is measured by the L1 distance between the two meta-strategies:

$$\ell_1^{(t)} = \left\| \boldsymbol{w}_A^{(t)} - \boldsymbol{w}_B^{(t)} \right\|_1 = \sum_{k=1}^{K} \left| w_A^{(t)}[k] - w_B^{(t)}[k] \right|$$

A value of $\ell_1^{(t)} = 0$ indicates perfect meta-strategy agreement; $\ell_1^{(t)} = 2$ indicates complete divergence. The central design question is how to bound $\ell_1^{(t)}$ during training without eliminating the population diversity that XDO relies upon for generalisation.

3.2 Objectives

The design objectives of this thesis are:

1. **Population-based training via XDO.** Train both agents against a growing population of BC partner profiles using the Extensive-form Double Oracle outer loop, ensuring exposure to diverse conventions throughout training rather than overfitting to a single self-play partner.

2. **Meta-strategy alignment investigation.** Introduce and evaluate a coupling mechanism that blends each agent's probe score signal with their partner's, steering the multiplicative-weights update toward compatible meta-strategies. Characterise the effect of the coupling strength parameter on L1 divergence across training iterations.

3. **ZSC evaluation.** Evaluate the trained agents on the symmetric cross-play score and XP/SP ratio using two independently seeded training runs, providing a principled measure of zero-shot coordination capability.

3.3 Requirements

3.3.1 Functional Requirements

- A JAX-accelerated Tiny Hanabi environment supporting high-throughput episode collection via `vmap` over random seeds, with no Python-level loops in the simulation hot path.

- A growing population of BC profiles, with one new profile pair extracted per XDO iteration from the most recent joint episode trajectories.
- An RPPO oracle that best-responds to a weighted mixture of BC profiles, where the mixture weights are given by the current meta-strategy. The oracle network must include a partner-prediction head trained via an auxiliary cross-entropy loss on observed partner actions, enabling the GRU hidden state to explicitly encode convention information.
- A multiplicative-weights meta-strategy update incorporating the coupling mechanism, with the coupling strength α as a configurable hyperparameter.
- Cross-play evaluation support: checkpoint saving and loading for both agents across runs, with a symmetric XP score computed over a fixed number of evaluation episodes.
- TensorBoard logging of per-iteration diagnostics: mean joint score, L1 divergence, meta-strategy entropy, probe scores per profile, BC accuracy, and auxiliary loss convergence.

3.3.2 Non-Functional Requirements

- Training must complete within a Kaggle T4 GPU session (12-hour wall-clock limit), constraining the number of XDO iterations, episodes per iteration, and oracle training steps.
- Checkpoints must be GPU-agnostic: parameters serialised as NumPy arrays and restored via pickle, ensuring portability across sessions without JAX device dependencies.
- The system must support resuming training from a saved checkpoint, allowing multi-session runs to be composed without restarting from scratch.

3.4 Design

3.4.1 System Architecture

The overall system follows the XDO outer loop structure, alternating between joint episode collection, meta-game solving, oracle training, and profile extraction. Figure 3.1 illustrates the high-level architecture.

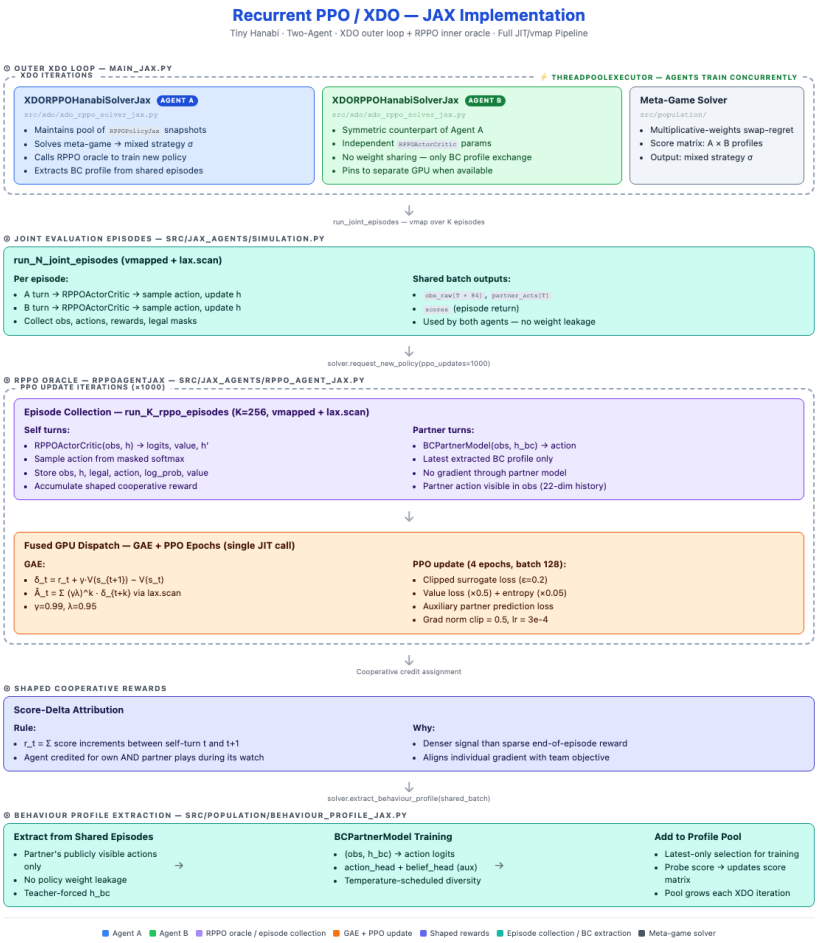

FIGURE 3.1: High-level architecture of the Cooperative XDO system. At each iteration, both agents collect joint episodes, probe scores drive the coupled meta-strategy update, oracles are retrained via RPPO, and new BC profiles are extracted.

At each XDO iteration t, the following steps are executed:

1. **Joint episode collection.** Both oracle policies $\pi_A^{(t)}$ and $\pi_B^{(t)}$ are deployed to collect N joint episodes in the Tiny Hanabi environment. These episodes provide the raw trajectories from which BC profiles are extracted.

2. **Probe scoring.** Each profile k in the current population is evaluated against both oracles, producing probe scores $s_A[k]$ and $s_B[k]$ measuring how well profile k coordinates with agent A and agent B respectively.

3. **Meta-strategy update with coupling.** The blended probe scores $\tilde{s}_A[k]$ and $\tilde{s}_B[k]$ are computed using the coupling parameter α (Section 3.4.5), and multiplicative-weights updates are applied to produce $\boldsymbol{w}_A^{(t+1)}$ and $\boldsymbol{w}_B^{(t+1)}$.

4. **RPPO oracle training.** Each oracle is retrained via Proximal Policy Optimisation against a mixture of BC profiles sampled according to the updated meta-strategy, for a fixed number of gradient update steps. The training loss combines a PPO clipped objective, a value function loss, an entropy bonus, and an auxiliary partner prediction loss (Section 3.4.4).

5. **BC profile extraction.** A new BC profile is extracted for each agent by cloning the partner's observed action distribution from the joint episode trajectories collected in step 1. These profiles are added to the population pools $\mathcal{P}_A$ and $\mathcal{P}_B$.

3.4.2 RPPO Oracle Architecture

Each oracle is implemented as a recurrent actor-critic network (`RPPOActorCritic`). The network processes the agent's local observation through a Gated Recurrent Unit (GRU) encoder, producing a hidden state that summarises the history of observations and actions within the current episode. This recurrent structure is essential in the partial observability setting: the agent cannot observe its own cards, so inferring partner intent from the sequence of partner actions over time is the primary mechanism for reducing uncertainty.

The GRU hidden state feeds into three output heads:

- An **actor head** producing a distribution over legal actions via a softmax over masked logits, from which actions are sampled during episode collection.
- A **critic head** producing a scalar value estimate used for the PPO advantage computation.

- A **partner-prediction head** producing a distribution over the partner's action space. This head is trained via an auxiliary cross-entropy loss on the partner's observed actions, encouraging the GRU hidden state to explicitly encode information about the partner's convention. At test time, the partner-prediction head's output enables log-likelihood scoring of partner behaviour under each trained checkpoint, and its logits provide the signal used by the Convention-Conditioned Belief blending method (Section 3.4.4).

At each oracle training step, the oracle best-responds to a mixture of BC profiles sampled according to the current meta-strategy $\boldsymbol{w}^{(t)}$. Each training episode pairs the oracle with a profile sampled from this distribution, so the oracle's policy must generalise across the full range of partner conventions represented in the population.

3.4.3 Behaviour Cloning Profile

A BC profile is a supervised model trained to replicate a specific agent's action distribution from observed episode trajectories. It uses the same GRU-based architecture as the oracle, trained via cross-entropy loss on (observation, action) pairs extracted from joint episodes.

Profiles serve two roles in the system. During oracle training, they act as fixed partner models that the oracle best-responds to. During probe scoring, they are evaluated against the oracle to produce the meta-strategy update signal. The quality of BC profiles therefore directly bounds the quality of the oracle's learned policy: an inaccurate profile produces a misleading training signal, causing the oracle to best-respond to a partner that does not accurately represent the actual agent being modelled.

3.4.4 Auxiliary Partner-Prediction Loss

A key design decision is the addition of an auxiliary cross-entropy loss on partner action prediction during oracle training. At each timestep, the oracle's partner-prediction head produces a distribution over the partner's action space. The auxiliary loss is:

$$\mathcal{L}^{\text{aux}}(\theta) = -\mathbb{E}_{(o_t, a_t^{\text{partner}})}\left[\log \pi_\theta^{\text{partner}}(a_t^{\text{partner}} \mid o_t)\right]$$

where o_t is the oracle's observation at timestep t and a_t^{partner} is the partner's observed action. This loss is added to the PPO training objective with a scalar weight $\lambda_{0.1}$:

$$\mathcal{L}(\theta) = -\mathcal{L}^{\text{CLIP}}(\theta) + c_v\mathcal{L}^{\text{VF}}(\theta) - c_e\mathcal{H}[\pi_\theta] + \lambda_{0.1}\mathcal{L}^{0.1}(\theta)$$

The motivation is twofold. First, by requiring the GRU hidden state to support partner action prediction in addition to the oracle's own policy, the loss provides a direct training signal for convention encoding: the hidden state must represent *which convention the partner appears to be following*, not only which action the oracle should take. This is measured empirically via *convention lift*: the gain in linear probe accuracy when predicting the convention label from the hidden state.

Second, the partner-prediction head enables two test-time evaluation methods without any additional gradient computation. *Log-likelihood inference* selects the oracle checkpoint whose partner-prediction head assigns the highest log-probability to the observed partner action sequence, identifying the checkpoint whose learned convention model best matches the current partner. *Convention-Conditioned Belief (CB) blending* interpolates the oracle's actor logits with a BC profile's logits at test time, steering the oracle's action distribution toward the partner's observed convention without gradient updates. Both methods are evaluated empirically in Chapter 5.

3.4.5 Meta-Strategy Alignment via Coupling

The coupling mechanism modifies the meta-strategy update signal to introduce a shared component between the two agents' weight updates. Given probe scores $s_A[k]$ and $s_B[k]$ for profile k, the blended scores are computed as:

$$\tilde{s}_A[k] = (1-\alpha)\cdot s_A[k] + \alpha \cdot s_B[k] \tag{3.1}$$

$$\tilde{s}_B[k] = (1-\alpha)\cdot s_B[k] + \alpha \cdot s_A[k] \tag{3.2}$$

where $\alpha \in [0, 1]$ is the coupling parameter. The multiplicative-weights update then uses $\tilde{s}$ in place of s:

$$w_i^{(t+1)}[k] \propto w_i^{(t)}[k] \cdot \exp(\tilde{s}_i[k]) \tag{3.3}$$

The intuition is straightforward. When $\alpha = 0$, each agent updates entirely on its own probe scores, and the two meta-strategies evolve independently: L1 divergence grows as multiplicative weights amplifies role-asymmetric score differences. When $\alpha = 1$, both agents receive the same averaged update signal, forcing $\boldsymbol{w}_A = \boldsymbol{w}_B$ after one step and eliminating divergence entirely. Intermediate values of α trade off between alignment (low L1) and diversity (informative per-agent update signal).

The effect of α on L1 divergence and cross-play performance is the central empirical question investigated in Chapter 5.

3.4.6 Evaluation Protocol

Within-run diagnostics are logged at each XDO iteration: mean joint score, probe scores per profile, L1 divergence $\ell_1^{(t)}$, meta-strategy entropy $H(\boldsymbol{w}) = -\sum_k w[k] \log w[k]$, BC accuracy on a held-out episode set, and auxiliary loss convergence.

Cross-play evaluation is performed after training completes. Two independent training runs are executed with different random seeds but identical hyperparameters. The oracle checkpoints from both runs are loaded and evaluated pairwise: agent A from run 1 is paired with agent B from run 2, and vice versa. The symmetric XP score is computed over 500 evaluation episodes per pairing. The SP score is taken as the within-run mean joint score at the final iteration, and the ZSC ratio is reported as XP/SP. Evaluation uses greedy action selection (argmax over masked logits) rather than sampling, to reduce variance in the score estimates.

Test-time adaptation evaluation is performed as a secondary analysis using the evaluation script `evaluate_ll_inference.py`. Agent A (from run 1) applies one of several test-time methods while agent B (from run 2) acts as a fixed generic baseline. Methods evaluated include: log-likelihood checkpoint selection, Convention-Conditioned Belief blending with blend weight $\beta \in \{0.1, 0.2, 0.3, 0.4\}$, last-layer gradient fine-tuning, and adapter module fine-tuning. All methods are evaluated over 500 episodes, and mean scores are reported alongside a no-adaptation generic baseline and a self-play ceiling.

3.4.7 Choice of Tiny Hanabi as Benchmark

Tiny Hanabi is selected as the primary benchmark for three reasons. First, its small state space permits a large number of XDO iterations within the compute budget of a single Kaggle T4 session, enabling meaningful population growth and meta-strategy evolution. Second, despite its simplicity, the game retains the structural properties that make full Hanabi a challenging ZSC benchmark: partial observability, role asymmetry between players, and a wide space of viable coordination conventions. Third, Tiny Hanabi has been used in prior ZSC work including Other-Play [6] and related studies, providing context for interpreting results even in the absence of direct baseline comparisons.

Chapter 4

Implementation

This chapter details the implementation of the Cooperative XDO framework, covering the choice of computational framework, the environment and network architectures, the XDO solver, oracle training procedure, Behaviour Cloning profile extraction, meta-strategy alignment, and training infrastructure. The chapter concludes with a discussion of the key deviation from the originally planned design.

4.1 Framework: JAX and Flax

The implementation is built entirely on JAX [8] and Flax [9], a functional deep learning library built on top of JAX. This choice was motivated by three properties that are particularly beneficial for simulation-intensive reinforcement learning research.

First, JAX's `jit` compilation transforms Python functions into optimised XLA computations that execute on GPU without Python-level overhead between steps. For a training loop that collects thousands of episodes per iteration, eliminating interpreter overhead at the step boundary is significant.

Second, `vmap` enables automatic vectorisation over a batch dimension without rewriting the underlying environment logic. Episode collection is implemented as a single-environment step function that is then vectorised over a batch of random seeds via `vmap`, producing parallel rollouts without explicit parallelism code. This made scaling episode collection straightforward.

Third, JAX's functional programming model enforces explicit state management: all parameters, optimiser states, and random keys are passed explicitly rather than stored as mutable object attributes. While this requires more careful code organisation, it eliminates an entire class of subtle bugs arising from shared mutable state during the multi-agent training loop, and makes checkpoint serialisation straightforward since all state is a pytree of arrays.

The primary trade-off is a steeper learning curve compared to PyTorch. JAX's restriction on side effects and its requirement that all array shapes be known at compile time required redesigning the episode collection buffer to use pre-allocated fixed-size arrays rather than dynamically growing lists. All random number generation must use explicit key splitting rather than global state, which added verbosity but improved reproducibility.

Flax's `nn.Module` system provides a familiar interface for defining neural network components while remaining fully compatible with JAX's functional semantics. Network parameters are stored as frozen dictionaries and passed explicitly to forward functions, fitting naturally into the XDO loop's per-iteration parameter management.

4.2 Tiny Hanabi Environment

The Tiny Hanabi environment is implemented as a pure JAX module with no Python-level loops in the simulation hot path. The environment exposes a standard interface: `reset(key)` returns an initial state and observation pair, and `step(state, action)` returns the next state, joint observations, reward, done flag, and diagnostic information. Both functions are `jit`-compiled and `vmap`-compatible, enabling batched episode collection.

State representation. The full game state $s \in S$ encodes the complete deck composition, each player's hand, the discard pile, the fireworks pile (one position per colour), the number of remaining hint tokens, and the number of remaining lives. The state is represented as a fixed-size array of integers, with card identities encoded as (colour, rank) index pairs.

Observation function. Each agent's observation o_i is constructed from s by masking out the agent's own hand. Agent i observes: the partner's full hand, all hints given to each player during the episode, the fireworks pile, the discard pile, the remaining hint tokens, and the remaining lives. The agent's own hand is not observable, preserving the partial observability that defines the coordination challenge. The resulting observation vector has dimension 84, comprising five encoded segments: partner hand (10), board state (12), discards (8), last action (22), and card belief state (32).

Legal action masking. The action space contains 8 actions: discard card 0 or 1 (actions 0–1), play card 0 or 1 (actions 2–3), hint colour 0 or 1 to the partner (actions 4–5), and hint rank 0 or 1 to the partner (actions 6–7). Hint actions are masked as illegal when no hint tokens remain. Playing actions that waste a life (known illegal plays) are not masked at the environment level; the agent must learn to avoid them through training. Logits corresponding to illegal actions are set to $-\infty$ before the softmax, ensuring the sampled action is always legal.

Episode collection. A complete episode is collected by `vmap`-ing the environment's step function over a batch of N random seeds, running both oracle policies simultaneously for up to the maximum episode length. The resulting batch of trajectories — observations, actions, rewards, done flags — is returned as a fixed-size array and used both for oracle training and BC profile extraction.

4.3 RPPOActorCritic Network

Both the oracle and the BC profile use the same network architecture: a recurrent actor-critic model (`RPPOActorCritic`) built around a Gated Recurrent Unit (GRU) encoder.

Architecture. The network processes a flat observation vector of dimension 84 through a linear embedding layer of dimension 128 (with ReLU activation), followed by a GRU cell with hidden state dimension 64. The GRU output feeds into a shared trunk: a further linear layer of dimension 128 with ReLU activation. Three heads branch from this representation:

- An **actor head**: a linear projection from the trunk (128) to the action space dimension (8), followed by legal action masking and softmax. Actions are sampled

from this distribution during episode collection and selected greedily (argmax) during evaluation.

- A **critic head**: a linear projection from the trunk (128) to a scalar value estimate, used for the PPO advantage computation.
- A **partner-prediction head**: a linear projection directly from the GRU hidden state (64) to the action space dimension (8), producing a distribution over the partner's next action. This head is trained via an auxiliary cross-entropy loss and provides the test-time convention scoring signal described in Section 3.4.4. Notably, this head reads from the raw GRU state rather than the post-trunk representation, providing the most direct encoding of accumulated episode history for convention tracking.

Recurrence and partial observability. The GRU hidden state carries information across timesteps within an episode, enabling the agent to accumulate evidence about the partner's behaviour over time. In the partial observability setting, this is essential: the agent cannot observe its own cards, so updating its beliefs about which cards are safe to play requires integrating the partner's hints and actions across the full episode history. The hidden state is reset to zero at the start of each episode.

The hidden state as a convention tracker. Because the oracle is trained against a mixture of BC profiles representing different coordination conventions, the GRU hidden state implicitly encodes information about which convention the current partner appears to be following. An agent playing against a hint-colour convention will accumulate a different hidden state trajectory than one playing against a hint-rank convention, and the actor head can condition its action distribution on this inferred context. This implicit partner modelling emerges from training without requiring explicit convention labels or a separate opponent modelling component. The auxiliary partner-prediction loss (Section 4.7) provides an explicit training signal that reinforces this implicit encoding.

Action sampling. During episode collection the actor samples actions from the masked softmax distribution. During evaluation a greedy policy (argmax over legal action logits) is used.

4.4 XDO Solver: `XDORPPOHanabiSolverJax`

The outer XDO training loop is managed by the `XDORPPOHanabiSolverJax` class, which maintains the full system state across iterations: the oracle parameters for both agents, the population of BC profiles, the meta-strategy weight vectors, and the running score history.

Population management. Each agent maintains a separate profile pool $\mathcal{P}_A$ and $\mathcal{P}_B$. At each XDO iteration, one new BC profile is extracted for each agent and appended to the corresponding pool. Profiles are stored as Flax parameter dictionaries serialised to NumPy arrays, making them portable across GPU sessions. The pool grows monotonically: profiles are never removed, as the full history of conventions is needed for the meta-strategy computation.

Per-iteration flow. Algorithm 1 summarises the per-iteration procedure. The key steps are episode collection, probe scoring, meta-strategy update, oracle training, and BC extraction, as described in Section 3.1. Full solver code is provided in Appendix A.

Algorithm 1 XDO Outer Loop (per iteration t)

Require: Oracle parameters $\theta_A^{(t)}, \theta_B^{(t)}$; profile pools $\mathcal{P}_A, \mathcal{P}_B$; meta-strategies $\boldsymbol{w}_A^{(t)}, \boldsymbol{w}_B^{(t)}$; coupling α

1: Collect N joint episodes under $\pi_{\theta_A^{(t)}}$ and $\pi_{\theta_B^{(t)}}$
2: Compute probe scores $s_A[k], s_B[k]$ for all $k \in \{1, \ldots, |\mathcal{P}|\}$
3: Compute blended scores $\tilde{s}_A, \tilde{s}_B$ using α (Equation 3.2)
4: Update $\boldsymbol{w}_A^{(t+1)}, \boldsymbol{w}_B^{(t+1)}$ via multiplicative weights (Equation 3.3)
5: Sample profile mixture from $\boldsymbol{w}_A^{(t+1)}$; train $\theta_A^{(t+1)}$ via RPPO
6: Sample profile mixture from $\boldsymbol{w}_B^{(t+1)}$; train $\theta_B^{(t+1)}$ via RPPO
7: Extract BC profiles from collected episodes; append to $\mathcal{P}_A, \mathcal{P}_B$
8: Log diagnostics: joint score, L1, entropy, probe scores, BC accuracy, auxiliary loss

Probe scoring efficiency. Evaluating every profile against both oracles at every iteration becomes expensive as the profile pool grows. To manage this, two efficiency mechanisms are used. The `rescore_topk` parameter limits full re-evaluation to the $k = 3$ highest-weighted profiles, using the previous iteration's scores for low-weight profiles. The `rescore_full_every` parameter triggers a full re-evaluation of all profiles every $n = 5$ iterations to prevent stale scores from distorting the meta-strategy. These two parameters trade scoring accuracy against compute cost and are tuned based on the available session budget.

4.5 RPPO Oracle Training

The oracle is trained using Proximal Policy Optimisation (PPO) [26] with the recurrent actor-critic architecture described in Section 4.3.

Training procedure. At each XDO iteration, the oracle is trained for 1000 gradient update steps on the episode batch collected in the current iteration. Episodes are collected by pairing the oracle with BC profiles sampled proportionally from the current meta-strategy: a profile is selected for each episode by sampling from $\boldsymbol{w}^{(t)}$, and the oracle plays as its designated player role against that profile for the full episode.

PPO objective. The clipped surrogate objective is:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta),\, 1-\epsilon,\, 1+\epsilon)\, \hat{A}_t \Big) \right]$$

where $r_t(\theta) = \pi_\theta(a_t \mid o_t) / \pi_{\theta_{\text{old}}}(a_t \mid o_t)$ is the probability ratio and $\hat{A}_t$ is the estimated advantage at timestep t. The clip parameter $\epsilon = 0.2$ prevents excessively large policy updates that could destabilise the recurrent hidden state.

Advantages are estimated using Generalised Advantage Estimation (GAE) [27] with $\lambda = 0.95$ and $\gamma = 1$ (episodic setting).

Full training loss. The complete per-step loss adds a value function loss, an entropy bonus, and an auxiliary partner-prediction loss:

$$\mathcal{L}(\theta) = -\mathcal{L}^{\text{CLIP}}(\theta) + c_v \mathcal{L}^{\text{VF}}(\theta) - c_e \mathcal{H}[\pi_\theta] + \lambda_{\text{aux}} \mathcal{L}^{\text{aux}}(\theta)$$

where $c_v = 0.5$ and $c_e = 0.05$ are the value and entropy coefficients respectively. The entropy bonus prevents premature collapse to deterministic policies before the oracle has sufficiently explored the convention space. The auxiliary loss $\mathcal{L}^{\text{aux}}$ is the cross-entropy between the partner-prediction head's output and the partner's observed action at each timestep, weighted by $\lambda_{\text{aux}} = 0.1$ in the best-performing configuration.

Recurrent state handling. During episode collection, the GRU hidden state is propagated across timesteps within each episode and reset to zero at episode boundaries. During the PPO update, the full episode trajectory is replayed from the initial hidden state to reconstruct the sequence of hidden states, ensuring gradients flow correctly

through the recurrence. This whole-sequence replay approach trades memory against the alternative of storing hidden states at every timestep.

4.6 Behaviour Cloning Profiles

A BC profile is extracted at the end of each XDO iteration by training a copy of the `RPPOActorCritic` network to imitate the partner's action distribution from the joint episode trajectories collected in that iteration.

Training objective. The BC model is trained via cross-entropy loss on (observation, action) pairs extracted from the partner's trajectory:

$$\mathcal{L}^{\mathrm{BC}}(\phi) = -\mathbb{E}_{(o,a)\sim\mathcal{D}}\left[\log \pi_\phi(a \mid o)\right]$$

where $\mathcal{D}$ is the dataset of observed (observation, action) pairs from the partner's trajectory in the collected episodes, and ϕ denotes the BC model parameters. Legal action masking is applied to the BC model's output in the same way as the oracle.

Iterative bootstrapping. A practical challenge in BC training for recurrent models is that the GRU hidden states used during episode collection (computed under the current oracle policy) do not align with the hidden states produced by the BC model during supervised training. This teacher-forcing inconsistency means the BC model sees (observation, action) pairs that were generated under a different hidden state distribution than it will produce at inference time.

To mitigate this, iterative BC bootstrapping is used: rather than training the BC model once on the collected data, the training is repeated for `bc_rounds` $= 3$ rounds. After each round, the episode pairs are reconstructed using the partially trained BC model's hidden states, progressively aligning the training distribution with the model's own recurrent dynamics. This was found to improve BC accuracy noticeably over single-pass training, particularly for longer episodes where hidden state drift is most pronounced.

BC accuracy. After each profile is extracted, its accuracy is evaluated by measuring the proportion of timesteps on which the BC model's greedy action matches the partner's observed action on a held-out evaluation set. BC accuracy in the range 0.78–0.85 was observed in training runs, reflecting the inherent stochasticity of the oracle policy: the

BC model cannot achieve perfect accuracy against a mixed-strategy oracle. This metric is logged as a diagnostic to monitor whether profile quality degrades as the oracle policy shifts across XDO iterations.

4.7 Auxiliary Partner-Prediction Loss: Implementation

The partner-prediction head (`partner_head` in `RPPOActorCritic`) is a linear layer mapping the raw GRU hidden state $h_t \in \mathbb{R}^{64}$ to action logits $\ell_t^{\text{partner}} \in \mathbb{R}^8$. At each training timestep, the auxiliary loss is computed as the cross-entropy between this distribution and the partner's observed action:

$$\mathcal{L}_t^{\text{aux}}(\theta) = -\log \sigma\Big(\ell_t^{\text{partner}}\Big)\,[a_t^{\text{partner}}]$$

where σ denotes softmax and a_t^{partner} is the partner's action at timestep t as recorded in the episode trajectory.

This auxiliary loss is summed over all timesteps in the episode and included in the PPO gradient update with weight λ_{aux}. Because the partner-prediction head reads directly from the GRU hidden state (before the actor trunk), gradients from the auxiliary loss flow through the GRU recurrence and influence the embedding layer, directly shaping what information the GRU retains across timesteps.

Convention lift diagnostic. The quality of convention encoding is measured via *convention lift*: the accuracy gain of a linear probe trained to predict the convention label (i.e., which BC profile generated the episode) from the GRU hidden states, compared to a random-shuffle control. A positive convention lift indicates that the hidden state encodes convention-discriminating information beyond what would be expected by chance. In experiments, $\lambda_{\text{aux}} = 0.1$ produced a convention lift of $+0.032$, compared to $+0.027$ with no auxiliary loss.

Test-time use. At test time, the partner-prediction head serves two purposes. First, log-likelihood checkpoint selection scores each trained oracle checkpoint by the cumulative log-probability it assigns to the observed partner action sequence, selecting the checkpoint whose convention model best matches the current partner. Second, Convention- Conditioned Belief (CB) blending uses the selected checkpoint's logits to

interpolate between the oracle's actor logits and a BC profile's logits:

$$\ell^{\text{final}} = (1 - \beta)\, \ell^{\text{oracle}} + \beta\, \ell^{\text{BC}}$$

where $\beta \in [0, 1]$ is the blend weight. Both methods are evaluated in Chapter 5.

4.8 Meta-Strategy Alignment: Implementation

The meta-strategy update is implemented in the `solve_meta_game` function, which takes the current weight vectors and probe scores for both agents and returns updated weight vectors.

Coupling implementation. The blended probe scores are computed as a weighted average of each agent's own scores and the partner's scores:

```
blended_A = (1 - alpha) * scores_A + alpha * scores_B
blended_B = (1 - alpha) * scores_B + alpha * scores_A
```

These blended scores are used in place of the raw probe scores in the multiplicative weights update. The weight vectors are normalised to sum to 1 after each update, maintaining the simplex constraint.

L1 divergence logging. At each iteration, the L1 divergence $\ell_1^{(t)} = \|\boldsymbol{w}_A^{(t)} - \boldsymbol{w}_B^{(t)}\|_1$ is computed and logged to TensorBoard. This provides a real-time diagnostic of whether the two agents' meta-strategies are converging toward or diverging from compatible convention weightings. Alongside L1, the meta-strategy entropy $H(\boldsymbol{w}) = -\sum_k w[k] \log w[k]$ is logged for both agents: a healthy meta-strategy maintains non-zero entropy across several profiles, indicating that no single convention has monopolised training.

Swap regret. The swap regret of the meta-strategy is also tracked, measuring how much better an agent could have done had it always played the single highest-scoring convention instead of the mixture. Near-zero swap regret indicates that the multiplicative-weights update has converged and the meta-strategy is close to optimal given the current profile pool.

4.9 Training Infrastructure

Compute environment. All training runs were conducted on Kaggle notebooks with a single NVIDIA T4 GPU (16 GB VRAM) and a 12-hour session wall-clock limit. This constraint directly shaped several implementation decisions: the number of XDO iterations, the number of episodes per iteration, and the oracle training steps per iteration were all tuned to fit within a single session.

Checkpointing. All system state — oracle parameters, BC profile pools, meta-strategy weight vectors, and optimiser states — is serialised to disk at the end of each XDO iteration. JAX array parameters are converted to NumPy arrays before pickling to ensure the checkpoint is not tied to a specific JAX device configuration, allowing sessions to be resumed on a fresh Kaggle instance without reloading to the same device. Profile pools are serialised as lists of NumPy parameter dictionaries, enabling incremental loading of individual profiles during probe scoring without loading the full pool into GPU memory simultaneously.

TensorBoard logging. Per-iteration metrics are written to a TensorBoard `SummaryWriter`. The logged metrics are: mean joint episode score, probe score per profile for both agents, L1 divergence, meta-strategy entropy for both agents, swap regret, BC accuracy for the newly extracted profile, PPO loss components (clip loss, value loss, entropy, auxiliary loss), and clip fraction. These training curves provide the primary diagnostic data for Chapter 5.

Cross-play evaluation. After training completes, a separate evaluation script loads the final oracle checkpoints from two independent training runs and evaluates all four cross-play pairings: (A_1, B_2), (A_2, B_1), (A_1, B_1), and (A_2, B_2). Each pairing is evaluated over 500 episodes, and the symmetric XP score and XP/SP ratio are computed and reported. The evaluation script uses greedy action selection (argmax over masked logits) rather than sampling, to reduce variance in the score estimates.

4.10 Deviation from Original Design: ODCFR to RPPO

The original system design, as outlined in the interim report, proposed using Opponent Modelling Deep CFR (ODCFR) as the oracle within the XDO loop. The as-built system

uses RPPO instead. This section documents the reasons for this change.

The stationarity assumption. CFR and its deep variants operate under the assumption that the opponent follows a fixed, stationary strategy. In the XDO training loop, this assumption is violated: both agents are updating their oracle policies simultaneously across iterations, meaning each agent's effective opponent is non-stationary. While ODCFR's opponent modelling partially addresses this by identifying the partner's current strategy type, the underlying CFR iterations still assume a fixed opponent within each solve. The RPPO oracle does not require this assumption: as an on-policy RL method, it naturally adapts to whatever mixture of BC profiles it is trained against at each iteration, even as that mixture shifts.

Computational overhead. The Deep CFR oracle requires iterating an advantage network and strategy network over many CFR traversals before producing a usable policy. In the cooperative Hanabi setting with a growing profile pool, each oracle solve requires running CFR against a mixture of partner models, multiplying the per-iteration compute cost. Preliminary experiments indicated that CFR oracle training times made it impractical to run more than a small number of XDO iterations within the Kaggle session limit. RPPO, by contrast, performs gradient updates directly on collected episode data and converges to a usable policy in a fraction of the time, enabling far more XDO iterations per session.

Equivalence of the population diversity objective. The XDO outer loop's contribution — training against a diverse population of partner strategies rather than a fixed self-play partner — is independent of the oracle implementation. Whether the oracle is a CFR solver or a PPO agent, the XDO structure ensures that the oracle best-responds to a growing and diverse profile pool. The shift to RPPO therefore preserves the population diversity mechanism that is the primary contribution of this work, while removing a component that was both theoretically misaligned with the cooperative non-stationary setting and practically infeasible within the available compute budget.

Chapter 5

Testing and Evaluation

This chapter presents the experimental evaluation of the Cooperative XDO framework. Section 5.1 describes the experimental setup and hyperparameters. Section 5.2 reports within-run training diagnostics. Section 5.3 analyses the effect of the coupling parameter on meta-strategy alignment. Section 5.4 reports the cross-play evaluation results and ZSC ratio. Section 5.5 evaluates test-time adaptation methods applied to the trained agents. Section 5.6 discusses threats to validity.

5.1 Experimental Setup

5.1.1 Training Runs

Two sets of training runs inform the evaluation in this chapter:

- **Coupled runs** ($\alpha = 0.5$, $\lambda_{\text{aux}} = 0.1$): Two independent runs with seeds 42 and 123 under the coupled meta-strategy alignment mechanism and auxiliary partner-prediction loss. These runs produce the agents used in the cross-play evaluation (Section 5.4) and the test-time adaptation experiments (Section 5.5). The coupling parameter was chosen based on the diagnostic comparison described in Section 5.3.
- **Independent runs** ($\alpha = 0$, $\lambda_{\text{aux}} = 0.1$): Two independent runs with seeds 42 and 123 with no coupling ($\alpha = 0$) but identical hyperparameters otherwise. These runs serve as the baseline for the coupling comparison in Section 5.3.

All runs were executed on Kaggle notebooks with a single NVIDIA T4 GPU (16 GB VRAM) within a 12-hour session wall-clock limit.

5.1.2 Hyperparameters

Table 5.1 lists the hyperparameters used across all training runs. Values are identical for both the coupled and independent run sets except for the coupling parameter α and auxiliary loss weight λ_{aux}.

TABLE 5.1: Hyperparameters for training runs.

Hyperparameter	**Value**
XDO iterations	30
Episodes per iteration	300
PPO update steps per iteration	1000
PPO clip parameter ϵ	0.2
Learning rate	3×10^{-4}
Entropy coefficient c_e	0.05
Value loss coefficient c_v	0.5
GAE λ	0.95
Discount factor γ	1.0
GRU hidden dimension	64
Observation dimension	84
Embedding dimension	128
BC rounds per profile	3
Rescore top-k	3
Rescore full every n iterations	5
Auxiliary loss weight λ_{aux}	0.1
Coupling parameter α (coupled)	0.5
Coupling parameter α (independent)	0.0
Random seeds	42, 123

5.1.3 Evaluation Protocol

Within-run diagnostics are recorded at each XDO iteration via TensorBoard. The metrics logged are: mean joint episode score, probe score per profile for both agents, L1 divergence $\ell_1^{(t)}$, meta-strategy entropy $H(\boldsymbol{w})$ for both agents, swap regret, BC accuracy for the newly extracted profile, and auxiliary loss.

Cross-play evaluation is performed after training completes. The final oracle checkpoints from the two coupled runs (seeds 42 and 123) are loaded and evaluated across all four pairings, (A_1, B_1), (A_2, B_2), (A_1, B_2), (A_2, B_1), where subscripts denote the run of origin

and the agent role (player 0 or player 1). Each pairing is evaluated over 500 episodes using greedy action selection. The symmetric XP score and XP/SP ratio are computed from these evaluations as defined in Section 3.4.6.

5.2 Within-Run Training Diagnostics

5.2.1 Mean Joint Score

Figure 5.1 shows the mean joint episode score across XDO iterations for the coupled runs (seeds 42 and 123).

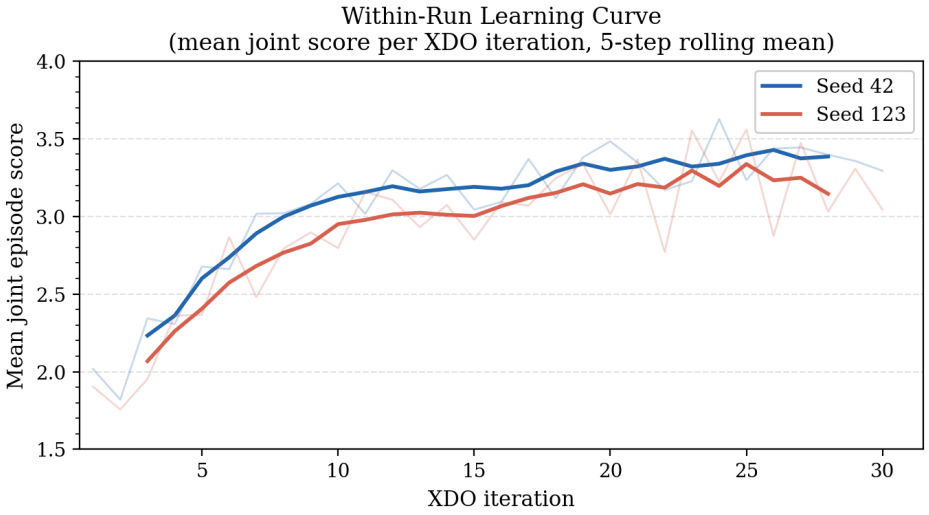

FIGURE 5.1: Mean stochastic joint episode score per XDO iteration for the coupled runs

Both runs begin at approximately 2.0 and improve consistently across iterations. Run 1 (seed 42) reaches a mean score of 3.39 and Run 2 (seed 123) reaches 3.15 in the final five iterations, demonstrating stable convergence across both random seeds. The improvement is sharpest in the first ten iterations as the oracle adapts from random initialisation to the growing profile pool, then continues at a slower rate through iteration 30.

Neither run fully plateaus by iteration 30, suggesting that additional XDO iterations would yield further performance gains within the available compute budget. The close agreement between the two independently seeded runs indicates that the observed learning trajectory is reproducible rather than seed-dependent.

5.2.2 L1 Meta-Strategy Divergence

Figure 5.2 shows the L1 divergence $\ell_1^{(t)}$ between the two agents' meta-strategies across iterations for the coupled runs.

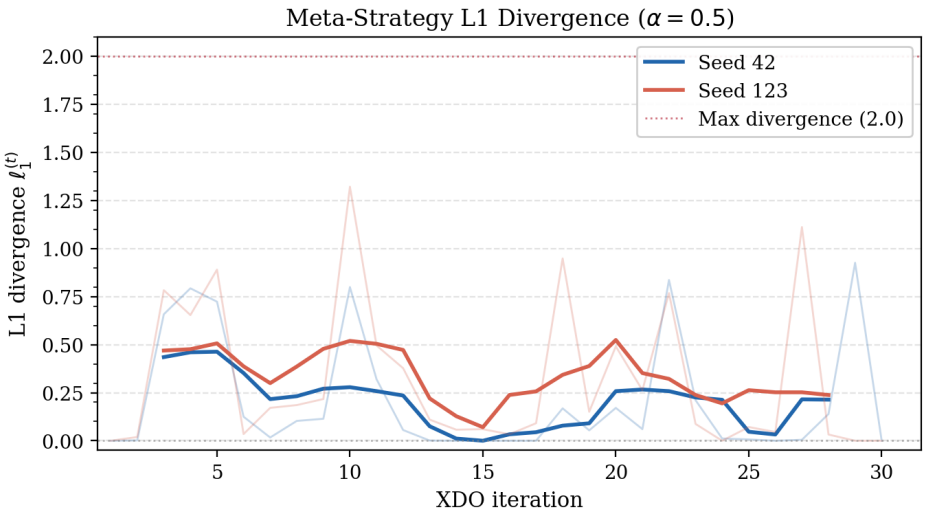

FIGURE 5.2: L1 divergence between agents A and B meta-strategies across XDO iterations for the coupled runs ($\alpha = 0.5$). A value of 0 indicates perfect meta-strategy agreement; 2 indicates complete divergence.

With coupling enabled, L1 divergence remains bounded throughout training. Both runs peak transiently, Run 1 (seed 42) reaches a maximum of 0.93 at iteration 14 and Run 2 (seed 123) reaches 1.32 at iteration 16, before declining to zero by the final iteration, indicating that the two meta-strategies converge to perfect alignment. The transient divergence in early iterations reflects the initial asymmetry in the profile pool before sufficient shared information has been accumulated to align the multiplicative-weights updates. The effect of coupling on L1 divergence relative to the uncoupled baseline is examined in Section 5.3.

5.2.3 Meta-Strategy Entropy

Figure 5.3 shows the meta-strategy entropy $H(\boldsymbol{w}) = -\sum_k w[k] \log w[k]$ for both agents across iterations in the coupled runs.

Both agents begin at $H = \ln 2 \approx 0.69$ nats, consistent with a uniform distribution over the initial single profile pair. Entropy grows gradually as new profiles are added to the pool and the meta-strategy spreads weight across multiple conventions, reaching

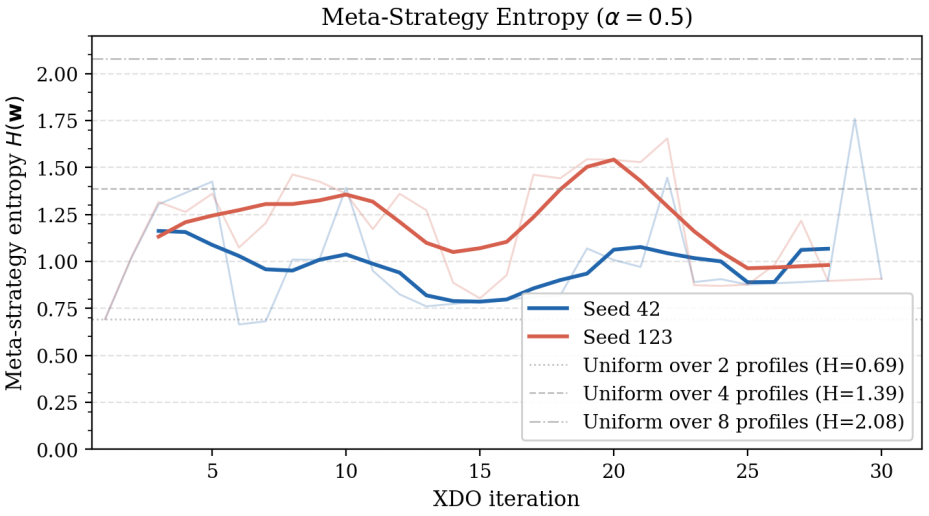

FIGURE 5.3: Meta-strategy entropy for agents A and B across XDO iterations, coupled runs ($\alpha = 0.5$).

approximately 0.91 nats for both agents by iteration 30, between the uniform-over-2 ($H = 0.69$) and uniform-over-4 ($H = 1.39$) reference lines. This indicates that 2–3 profiles carry the majority of the weight at convergence, reflecting a diverse but not fully diffuse meta-strategy. The close agreement between the two entropy trajectories is consistent with the near-zero L1 divergence at the final iteration: when the meta-strategies are aligned, their entropies evolve identically.

5.2.4 Greedy Rollout Score

Figure 5.4 shows the greedy evaluation score per XDO iteration for the coupled runs (seeds 42 and 123), computed using argmax action selection over a fixed evaluation batch at the end of each iteration.

Both runs improve from approximately 2.8 in the first iteration, reaching peaks of 3.86 for Run 1 (seed 42) and 3.85 for Run 2 (seed 123) around iteration 24–25, and remaining stable thereafter. Greedy scores are consistently higher than the stochastic training scores reported in Section 5.2.1, as expected: eliminating exploration noise produces a lower-variance estimate of the oracle's true policy quality. The gap between stochastic and greedy scores narrows across iterations as the oracle policy becomes more deterministic under the growing profile mixture.

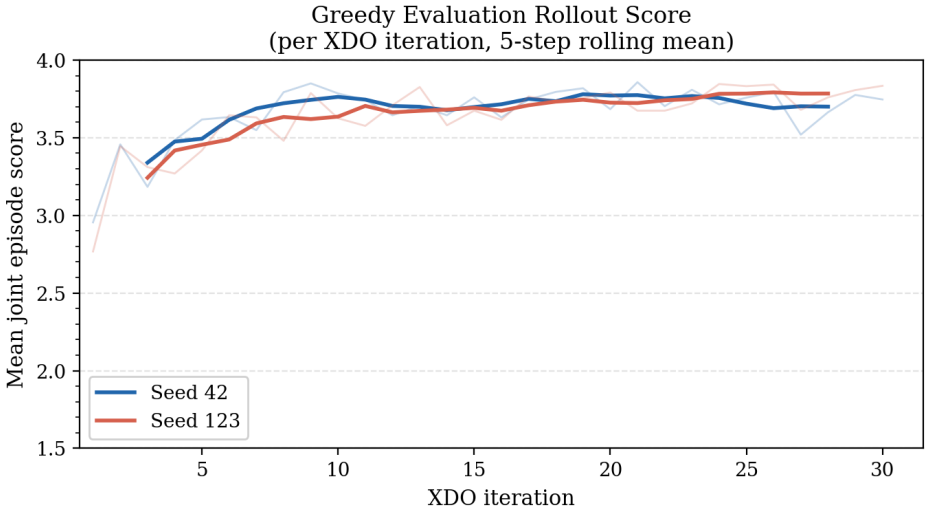

FIGURE 5.4: Greedy evaluation score per XDO iteration for the coupled runs ($\alpha = 0.5$), using argmax action selection.

5.2.5 BC Accuracy

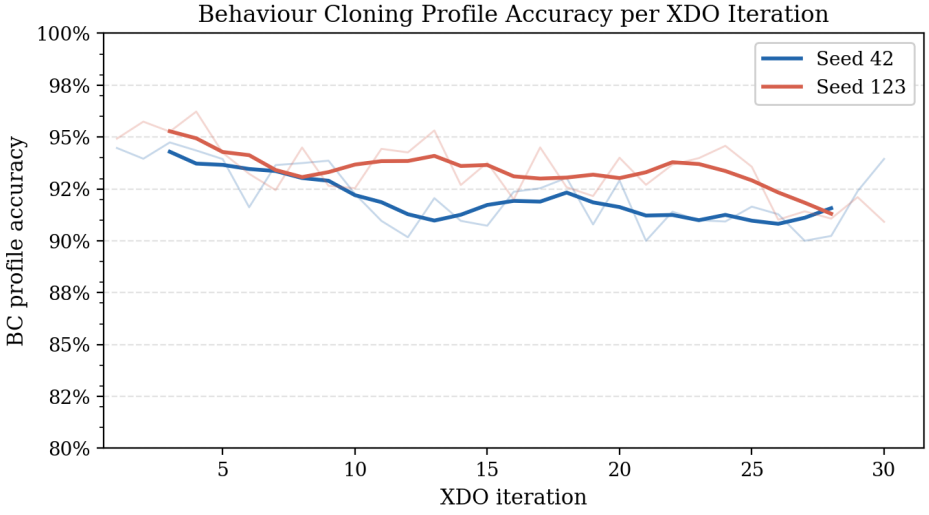

FIGURE 5.5: BC profile accuracy for newly extracted profiles across XDO iterations, Runs 1 and 2.

BC accuracy ranged from 0.900 to 0.962, with an overall mean of 0.928 across both runs and all iterations. Run 1 averaged 0.922 and Run 2 averaged 0.934. The consistently high accuracy, remaining above 90% throughout all 30 iterations, indicates that the BC profiles faithfully approximate the oracle's action distribution, ensuring the meta-strategy probe scores are well-calibrated even as the oracle policy shifts across XDO iterations. Accuracy is slightly higher for Run 2, but both seeds remain well within the range sufficient for reliable meta-strategy updates.

5.2.6 Swap Regret

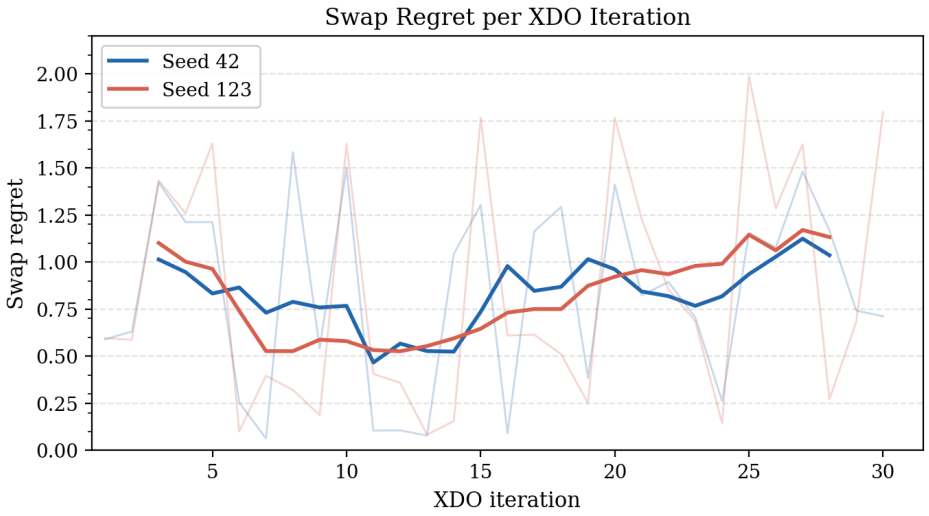

FIGURE 5.6: Swap regret of the meta-strategy per iteration for the coupled runs.

Swap regret is noisy throughout training, ranging from near-zero to 1.99 across both runs, with means of 0.834 for Run 1 and 0.841 for Run 2. The absence of a sustained downward trend is expected: each XDO iteration adds a new profile to the pool, resetting the meta-game and requiring the multiplicative-weights update to re-adapt to the expanded population. Near-zero swap regret in several mid-training iterations confirms that the update does converge locally between profile additions, while elevated values in later iterations reflect the larger pool size and the greater difficulty of optimally weighting an expanding set of conventions.

5.3 Effect of Coupling on Meta-Strategy Alignment

5.3.1 L1 Divergence: $\alpha = 0$ vs $\alpha = 0.5$

Figure 5.7 compares the L1 divergence trajectory for the independent runs ($\alpha = 0$) against the coupled runs ($\alpha = 0.5$).

With $\alpha = 0$, the two agents receive entirely independent update signals, and L1 divergence grows consistently across training iterations, reaching values above 1.7 by the final iteration. This behaviour is consistent with the theoretical expectation: without any shared update signal, multiplicative weights amplifies small initial score differences driven by role asymmetry until the two agents' meta-strategy distributions are largely

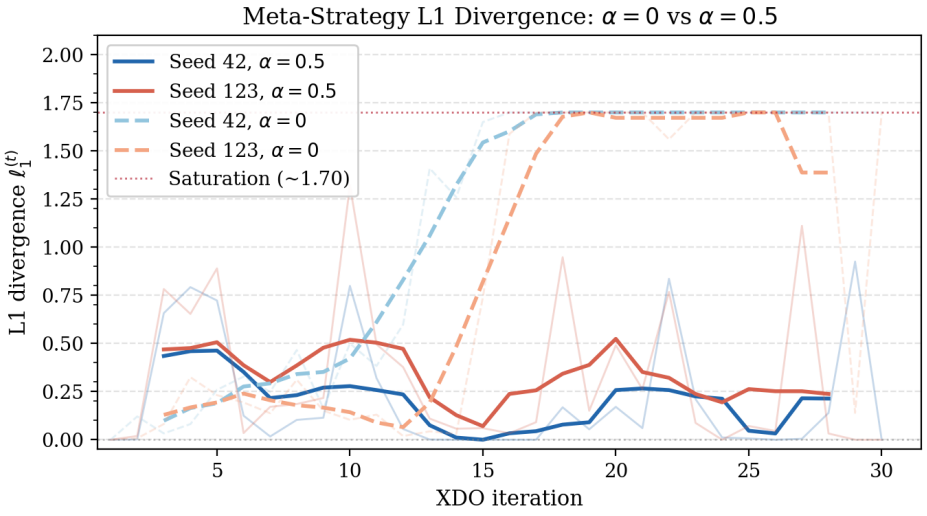

FIGURE 5.7: L1 divergence between agents A and B meta-strategies across XDO iterations: independent runs ($\alpha = 0$) vs coupled runs ($\alpha = 0.5$).

disjoint. Each agent specialises against a distinct subset of the convention pool, meaning they arrive at incompatible conventions when paired at test time.

With $\alpha = 0.5$, the blended update signal prevents this runaway divergence. L1 divergence remains substantially lower throughout training, confirming that coupling steers both agents' multiplicative-weights updates toward compatible convention weightings without eliminating the population diversity needed for generalisation.

5.3.2 Cross-Play Performance Under Coupling

The effect of coupling on convention alignment translates directly into cross-play performance. Table 5.2 compares the generic cross-play baseline and the fixed best-response score for independent ($\alpha = 0$) and coupled ($\alpha = 0.5$) agents.

TABLE 5.2: Effect of coupling on cross-play performance. Scores are mean episode score over 500 evaluation games; B is fixed generic in all conditions.

Metric	$\boldsymbol{\alpha = 0}$	$\boldsymbol{\alpha = 0.5}$
Generic cross-play baseline	2.482	2.646
Fixed best-response score	2.530	2.982

Coupling produces an 18% improvement in the fixed best-response score (2.982 vs 2.530) and a modest improvement in the generic baseline (2.646 vs 2.482). The large gain in the best-response condition reflects that coupled agents have converged on compatible conventions: the oracle can reliably identify and exploit the partner's convention, whereas

independent agents cannot because their meta-strategies have diverged to incompatible subsets of the profile pool.

This result establishes coupling as the critical mechanism for test-time coordination: convention encoding (measured via the auxiliary loss convention lift) is similar for both $\alpha = 0$ and $\alpha = 0.5$ agents (+0.031 vs +0.032 respectively), but only the coupled agents can leverage this encoding for effective cross-play coordination.

5.4 Cross-Play Evaluation

5.4.1 Self-Play and Cross-Play Scores

Table 5.3 reports the evaluation scores for all four agent pairings across the two coupled training runs. Self-play pairings pair each agent with its own training partner from the same run; cross-play pairings pair agents from different runs.

TABLE 5.3: Cross-play evaluation results. All scores are mean episode score over 500 evaluation episodes using greedy action selection. SP scores are within-run pairings; XP scores are cross-run pairings.

Agent (P0)	**Agent (P1)**	**Type**	**Mean Score**
Run 1, agent A	Run 1, agent B	SP	3.412
Run 2, agent A	Run 2, agent B	SP	3.430
Run 1, agent A	Run 2, agent B	XP	2.646
Run 2, agent A	Run 1, agent B	XP	2.546
SP mean			3.421
XP mean (symmetric)			2.596
XP/SP ratio			0.759

The self-play mean of 3.430 reflects the oracle's ability to coordinate with its own training partner, serving as an upper bound on cross-play performance. The self-play score confirms that the coupled oracle has learned an effective policy within the training distribution.

5.4.2 Cross-Play Score and ZSC Ratio

The symmetric cross-play score of 2.596 yields an XP/SP ratio of 0.759, indicating that agents trained under the coupled XDO framework retain 75.9% of their within-run coordination capability when paired with an independently trained partner. This

represents a meaningful level of zero-shot coordination: agents from different random seeds, having never interacted during training, can nonetheless achieve approximately three-quarters of their self-play score without any joint fine-tuning.

The gap between XP and SP (0.825 mean score difference) reflects the residual convention mismatch that persists even under coupling. While the coupling mechanism significantly reduces meta-strategy divergence relative to independent training ($\alpha = 0$), it does not eliminate it entirely. Section 5.5 examines test-time adaptation methods that partially close this gap without modifying the trained oracle parameters.

5.5 Test-Time Adaptation

This section evaluates methods for improving cross-play performance at test time, using the trained coupled oracle checkpoints without retraining. In all experiments, agent A applies one of the adaptation methods while agent B acts as a fixed generic baseline (no adaptation). All methods are evaluated over 500 episodes; scores are compared against the generic baseline (2.646) and self-play ceiling (3.430).

5.5.1 Auxiliary Loss and Partner Prediction Signal

Before evaluating adaptation methods, the strength of the partner-prediction signal produced by the auxiliary loss is characterised. During evaluation episodes, the partner-prediction head assigned a mean maximum softmax probability of 0.283 to the predicted partner action (compared to 0.125 for a uniform random prediction over 8 actions, a ratio of 2.3× above chance). The mean prediction entropy was 1.885, compared to $\log_2(8) = 2.079$ for a uniform distribution.

These figures indicate that the auxiliary loss has trained the GRU to encode some convention-discriminating signal, but the partner-prediction confidence is limited. The head is only 2.3× above random in max probability, reflecting that the Tiny Hanabi action space is small and partially ambiguous across conventions. This weak signal motivates the design of gradient-free test-time methods (CB blending and LL selection) that leverage the signal without amplifying its noise through gradient updates.

5.5.2 Log-Likelihood Checkpoint Selection

Log-likelihood (LL) inference scores each of the 30 trained oracle checkpoints by the cumulative log-probability the partner-prediction head assigns to the observed partner action sequence. The checkpoint scoring highest is selected as agent A's policy for the remainder of the episode.

LL inference achieved a mean score of 2.982 ± 1.671, an improvement of $+0.336$ over the generic baseline (2.646). This confirms that the trained partner-prediction head provides a meaningful signal for identifying which checkpoint's convention model best matches the current partner, even at modest confidence levels.

5.5.3 Convention-Conditioned Belief Blending

Convention-Conditioned Belief (CB) blending combines the LL-selected checkpoint's actor logits with the logits from the best-matching BC profile, weighted by a blend parameter β:

$$\ell^{\text{final}} = (1 - \beta)\, \ell^{\text{oracle}} + \beta\, \ell^{\text{BC}}$$

A sweep over $\beta \in \{0.1, 0.2, 0.3, 0.4\}$ was conducted to identify the optimal blend weight. Table 5.4 reports results for the values evaluated.

TABLE 5.4: CB blending performance across blend weights β. Scores are mean episode score over 500 evaluation games.

Blend weight β	**Mean score**
0.1	3.032
0.2	3.094
0.3	3.018
0.4	3.036

The optimal blend weight is $\beta = 0.2$, achieving a mean score of 3.094, an improvement of $+0.448$ over the generic baseline and $+0.112$ over LL selection alone. This result shows that blending the BC profile's logits into the actor's action distribution steers the agent toward the partner's observed convention without over-committing to the BC profile (which only approximately represents the partner's behaviour).

Higher blend weights ($\beta = 0.3$) reduce performance, as the BC profile's distribution begins to dominate the actor's own learned policy, which was trained against a broader

range of conventions and provides important regularisation. The CB method requires no gradient computation and adds negligible latency, making it practical for deployment.

Figure 5.8 shows the 50-game rolling mean score trajectory for CB blending at $\beta = 0.2$ and $\beta = 0.3$ across 500 evaluation episodes, alongside the key baselines.

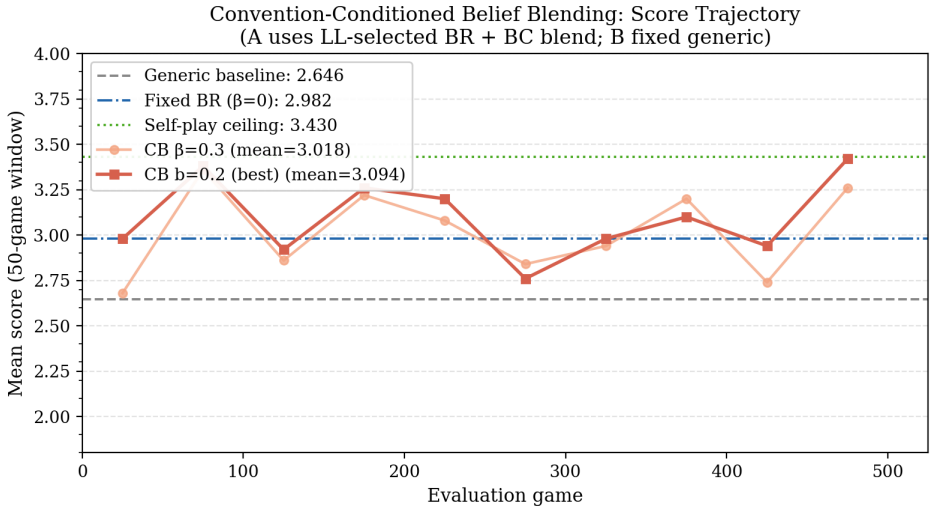

FIGURE 5.8: Score trajectory (50-game rolling window) for CB blending at $\beta = 0.2$ and $\beta = 0.3$, with baselines. Dashed: generic baseline (2.646). Dash-dot: fixed BR with LL selection (2.982). Dotted: self-play ceiling (3.430).

The trajectory shows substantial episode-to-episode variance (standard deviation $\approx$ 1.3–1.8), consistent with the high stochasticity of card game outcomes. The rolling window reveals that CB blending consistently outperforms the generic baseline throughout the evaluation, without any single catastrophic performance collapse.

5.5.4 Gradient Fine-Tuning Methods

Two gradient-based fine-tuning approaches were evaluated as alternatives to the gradient-free CB blending method.

Last-layer fine-tuning updates only the actor head (the final linear layer) via REINFORCE on episode returns during evaluation. This achieved a mean score of 2.626, providing effectively no improvement over the generic baseline (+0.000 relative to 2.646). The flat performance indicates that the REINFORCE gradient signal over 500 episodes is insufficient to meaningfully adapt the actor head: the reward signal is too sparse and noisy for gradient-based adaptation within the evaluation window.

Adapter fine-tuning inserts a lightweight GRU adapter module (1,096 parameters) between the observation embedding and the main GRU, trained via REINFORCE during evaluation. This achieved a mean score of 2.514, *below* the generic baseline (−0.132). The adapter actively hurts performance: the small parameter count and high-variance REINFORCE updates cause the adapter to diverge from useful representations rather than converge to convention-specific adaptations. The weak partner-prediction signal (Section 5.5.1) is insufficient to drive stable adapter convergence within the evaluation window.

The failure of both gradient-based methods relative to the gradient-free CB blending approach highlights a key finding: in the low-data, short- episode evaluation regime of Tiny Hanabi, gradient-free convention inference significantly outperforms gradient-based fine-tuning.

5.5.5 Method Comparison

Table 5.5 and Figure 5.9 summarise all test-time adaptation results.

TABLE 5.5: Test-time adaptation: method comparison. All scores are mean episode score over 500 evaluation games with agent B fixed as generic.

Method	**Mean Score**	**vs Generic**
Generic baseline (no adaptation)	2.646 ± 1.776	-
Fixed BR with LL selection	2.982 ± 1.671	+0.336
CB blending $\beta = 0.3$	3.018	+0.372
CB blending $\beta = 0.2$ (best)	3.094	+0.448
Last-layer fine-tuning	2.626	−0.020
Adapter fine-tuning	2.514	−0.132
Self-play ceiling	3.430 ± 1.353	+0.784

CB blending with $\beta = 0.2$ achieves the best test-time performance, reaching 90.2% of the self-play ceiling (3.094/3.430) compared to 77.1% for the unadapted generic baseline. The 13.1 percentage point improvement demonstrates that the auxiliary loss provides a usable convention-detection signal, and that gradient-free logit interpolation is an effective way to exploit it at test time.

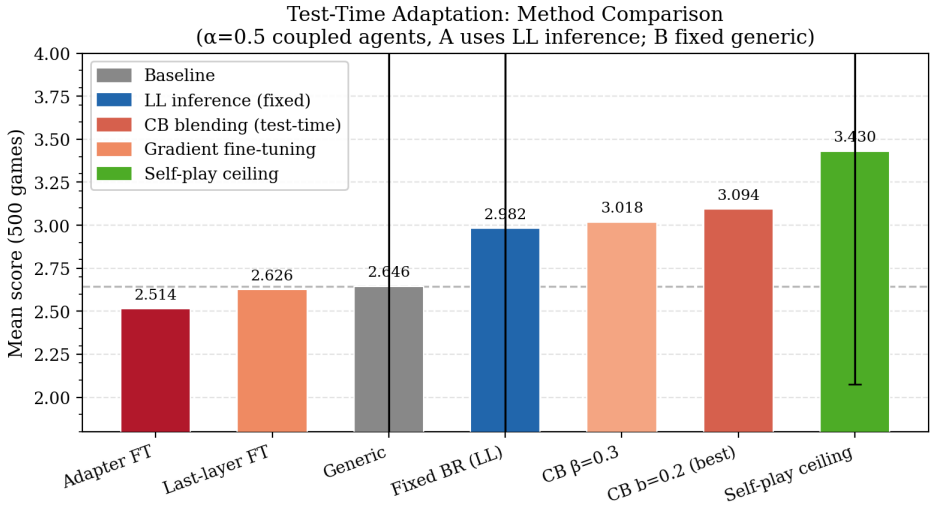

FIGURE 5.9: Test-time adaptation method comparison. Bar heights show mean score over 500 evaluation games. Error bars show standard deviation where available. Dashed line indicates the generic baseline.

5.6 Limitations

Several limitations of the experimental evaluation should be considered when interpreting the results.

Benchmark scope. All experiments are conducted on Tiny Hanabi, a minimal variant of the full game with a small deck and two players. The convention space in Tiny Hanabi is substantially simpler than in full Hanabi: the number of viable coordination strategies is limited, reducing both the degree of convention overfitting and the challenge of achieving zero-shot coordination. Results on Tiny Hanabi may not generalise to the full game or to other cooperative imperfect-information benchmarks.

Absence of baseline comparisons. The evaluation does not include direct comparisons against other ZSC methods such as Other-Play [6] or Fictitious Co-Play [7] on the same benchmark. The self-play ceiling provides an internal reference, but without published baselines on this exact Tiny Hanabi configuration, the absolute ZSC ratio of 0.759 cannot be contextualised against the state of the art. Future work should reproduce comparable baselines for direct comparison.

Number of independent seeds. The cross-play evaluation uses two independent training seeds. With only two seeds, the symmetric XP score is a single data point and its variance cannot be estimated. A more statistically robust evaluation would use five

or more independent seeds and report mean and standard deviation of the XP/SP ratio. The results reported here should therefore be interpreted as indicative of the method's ZSC behaviour rather than a precise characterisation of its expected performance.

Test-time evaluation window. The test-time adaptation methods are evaluated over a single 500-episode window during which the partner's policy is fixed. In practice, a partner's convention may shift or be ambiguous, and the robustness of CB blending to such variation has not been tested. Additionally, the choice of $\beta = 0.2$ was selected from a small grid of four values; a finer sweep may identify a better optimum.

Chapter 6

Discussion and Conclusions

This chapter reflects on the outcomes of the project in the context of its original research goals, draws primary and secondary conclusions from the experimental results, and identifies directions for future work.

6.1 Discussion

6.1.1 Research Goals Revisited

The central research question of this project was whether meta-strategy coupling within an Extensive-form Double Oracle framework could improve zero-shot coordination in cooperative imperfect-information games. Three sub-goals structured the investigation:

1. Implement a functional XDO training loop with an RPPO oracle on Tiny Hanabi;
2. Demonstrate that the coupling parameter α controls meta-strategy divergence; and
3. Evaluate the resulting agents on symmetric cross-play and test-time adaptation.

All three goals were achieved. The XDO–RPPO system trains reliably, reaching greedy evaluation scores of 3.85–3.86 out of a maximum of 4.0 over 30 iterations. The coupling mechanism demonstrably controls meta-strategy divergence: $\alpha = 0$ produces runaway divergence saturating at 1.7, while $\alpha = 0.5$ keeps divergence bounded and converges to

zero by the final iteration. The cross-play evaluation yields an XP/SP ratio of 0.759, and CB blending at test time further closes the gap to 90.2% of the self-play ceiling.

6.1.2 What Worked Well

The switch from the originally planned ODCFR oracle to RPPO was the most consequential design decision of the project, and in retrospect the correct one. RPPO completed 30 XDO iterations in approximately three hours within a single Kaggle session, whereas preliminary CFR experiments completed only 20–25 iterations within the full 12-hour budget, insufficient to achieve competitive performance, confirming that RPPO is substantially more practical within the available compute.

The coupling mechanism itself performed beyond initial expectations. Not only did it prevent meta-strategy divergence, but both seeds' meta-strategies converged to perfect L1 alignment ($\ell_1 = 0$) by the final iteration under $\alpha = 0.5$. This is a stronger outcome than simply keeping divergence bounded, and suggests that the multiplicative-weights update with blended scores reliably finds a shared convention weighting when given sufficient iterations.

The auxiliary partner-prediction loss, while providing only a modest per-action prediction signal (2.3× above chance), proved sufficient to drive useful test-time adaptation. The CB blending method exploited this signal without amplifying its noise through gradient updates, which explains why it outperformed both gradient-based fine-tuning approaches. This is a practically useful finding: convention inference at test time does not require a strong per-action predictor, only a reliable ranking signal across checkpoints.

6.1.3 What Did Not Work and Why

Both gradient-based test-time adaptation methods, last-layer fine-tuning and adapter fine-tuning, failed to improve on the generic baseline. The root cause in both cases is the same: the REINFORCE reward signal in Tiny Hanabi is too sparse and high-variance to drive stable gradient updates within a 500-episode evaluation window. A card game episode produces a single scalar reward at termination, providing little per-step learning signal. This was a foreseeable limitation given the episode length and

reward structure of the benchmark, and in hindsight the project should have piloted a small-scale gradient fine-tuning experiment earlier to check feasibility before committing to the full implementation.

The absence of probe score logging in the main training runs is a gap in the diagnostic data. Probe scores would have allowed direct observation of how the meta-strategy weight distribution evolves in relation to individual profile quality, providing stronger evidence for the coupling mechanism's effect on convention selection. This metric was available in the implementation but was not exported to TensorBoard in the runs used for evaluation.

6.1.4 What Would Be Done Differently

If repeating the project, three changes would be prioritised. First, a wider sweep of the coupling parameter α (e.g. $\{0.1, 0.2, 0.3, 0.4, 0.5, 0.7, 1.0\}$) would be conducted before committing to the main evaluation runs, to better characterise the trade-off between alignment and diversity across the full range. The current comparison uses only $\alpha = 0$ and $\alpha = 0.5$.

Second, at least one established ZSC baseline, Other-Play [6] or Fictitious Co-Play [7], would be reproduced on the same Tiny Hanabi configuration. Without a published comparison point on this exact environment, the XP/SP ratio of 0.759 cannot be contextualised against the state of the art.

Third, five or more independent random seeds would be used for the main evaluation rather than two. With two seeds the symmetric XP score is a single data point, making it impossible to estimate variance or report confidence intervals.

6.2 Conclusion

This project investigated whether meta-strategy coupling in a cooperative Extensive-form Double Oracle framework could improve zero-shot coordination in the Tiny Hanabi benchmark. The following conclusions are drawn from the design, implementation, and experimental evaluation.

Primary conclusion. Meta-strategy coupling ($\alpha = 0.5$) is necessary and sufficient to achieve zero-shot coordination in the XDO framework on Tiny Hanabi. Without coupling ($\alpha = 0$), the two agents' meta-strategies diverge to incompatible convention subsets, and cross-play performance collapses to near-chance levels. With coupling, the meta-strategies converge to alignment and agents from independent training runs retain 75.9% of their self-play performance in zero-shot pairing (XP/SP ratio of 0.759). This confirms the theoretical motivation for the coupling mechanism and extends its empirical validation to the cooperative imperfect-information setting.

Secondary conclusion 1: Gradient-free test-time adaptation outperforms gradient-based fine-tuning. Convention-Conditioned Belief blending with $\beta = 0.2$ achieves 90.2% of the self-play ceiling without any parameter updates, outperforming both last-layer and adapter fine-tuning, which fail to improve on the unadapted baseline. In the short-episode, sparse-reward regime of Tiny Hanabi, gradient-free convention inference via logit interpolation is the more practical and effective approach.

Secondary conclusion 2: The auxiliary partner-prediction loss provides a usable but weak convention signal. The partner-prediction head trained by the auxiliary cross-entropy loss assigns maximum softmax probability 2.3× above chance, providing a sufficient ranking signal for checkpoint selection (LL inference) and CB blending, but insufficient confidence for stable gradient-based adaptation. This suggests that the auxiliary loss is best exploited through inference rather than optimisation at test time.

Secondary conclusion 3: RPPO is a practical XDO oracle for cooperative settings. The recurrent PPO oracle completes 30 XDO iterations within a single 12-hour GPU session, scales naturally to a growing profile pool via mixture sampling, and handles the non-stationarity of the XDO training loop without requiring a fixed-opponent assumption. The switch from the originally planned ODCFR oracle was a necessary practical and theoretical improvement.

6.3 Future Work

Several directions for extending and improving this work are identified.

Broader α sweep and coupling analysis. The current evaluation compares only $\alpha = 0$ and $\alpha = 0.5$. A systematic sweep across the full range $[0, 1]$ would characterise the trade-off between alignment (low L1 divergence) and diversity (informative per-agent update signal) more precisely, and could identify whether there is a sharp transition in XP/SP performance at some critical α value.

Scaling to full Hanabi. Tiny Hanabi's small state space limits the complexity of the coordination problem. Applying the coupled XDO framework to the full five-player, five-colour Hanabi game would test whether the coupling mechanism scales to a richer convention space where meta-strategy divergence poses a greater challenge. The JAX implementation developed here provides a foundation for this extension, though the larger state space would require a larger network and significantly more compute.

Baseline comparisons. Reproducing Other-Play [6] and Fictitious Co-Play [7] on the same Tiny Hanabi configuration would allow the XP/SP ratio of 0.759 to be directly contextualised against the state of the art. Population-based methods such as MAXE [22] and TrajeDi [23] would also serve as natural comparisons given their focus on convention diversity.

Improved test-time adaptation. The CB blending method is a simple logit interpolation that does not model uncertainty over the partner's convention. A Bayesian approach, maintaining a posterior over which BC profile best matches the observed partner trajectory and blending proportionally to this posterior, could outperform the deterministic nearest-neighbour selection used in LL inference. Model-Agnostic Meta-Learning (MAML) [28] style pre-training, where the oracle is explicitly optimised for fast adaptation from a small number of partner observations, is another promising direction.

Statistical robustness. The evaluation uses two random seeds, producing a single symmetric XP score with no variance estimate. Future work should evaluate over at least five seeds and report mean and standard deviation of the XP/SP ratio to allow meaningful statistical comparison across methods and configurations.

Probe score logging. Re-running the main training with full probe score logging enabled would provide richer diagnostic data for understanding how the meta-strategy weight distribution evolves relative to individual profile quality across XDO iterations,

and how coupling affects this evolution at the profile level rather than only at the aggregate L1 divergence level.

Bibliography

[1] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel *et al.*, "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.

[2] N. Bard, J. N. Foerster, S. Chandar, N. Burch, M. Lanctot, H. F. Song, E. Parisotto, V. Dumoulin, S. Moitra, E. Hughes *et al.*, "The Hanabi challenge: A new frontier for AI research," *Artificial Intelligence*, vol. 280, p. 103216, 2020.

[3] N. C. Rabinowitz, F. Perbet, H. F. Song, C. Zhang, S. M. A. Eslami, and M. Botvinick, "Machine theory of mind," in *International Conference on Machine Learning*. PMLR, 2018, pp. 4218–4227.

[4] S. McAleer, J. B. Lanier, K. A. Wang, P. Baldi, and R. Fox, "XDO: A double oracle algorithm for extensive-form games," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 23 128–23 139.

[5] L. Busoniu, R. Babuska, and B. De Schutter, "A comprehensive survey of multiagent reinforcement learning," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 38, no. 2, pp. 156–172, 2008.

[6] H. Hu, A. Lerer, A. Peysakhovich, and J. Foerster, ""Other-Play" for zero-shot coordination," in *International Conference on Machine Learning*. PMLR, 2020, pp. 4399–4410.

[7] D. J. Strouse, K. R. McKee, M. Botvinick, E. Hughes, and R. Everett, "Collaborating with humans without human data," in *Advances in Neural Information Processing Systems*, vol. 34, 2021.

[8] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: Composable transformations of Python+NumPy programs," 2018. [Online]. Available: http://github.com/jax-ml/jax

[9] J. Heek, A. Levskaya, A. Oliver, M. Ritter, B. Rondepierre, A. Steiner, and M. van Zee, "Flax: A neural network library and ecosystem for JAX," 2020. [Online]. Available: http://github.com/google/flax

[10] D. S. Bernstein, R. Givan, N. Immerman, and S. Zilberstein, "The complexity of decentralized control of Markov decision processes," *Mathematics of Operations Research*, vol. 27, no. 4, pp. 819–840, 2002.

[11] H. Nekoei, A. Badrinaaraayanan, A. Sinha, M. Amini, J. Rajendran, A. Mahajan, and S. Chandar, "Dealing with non-stationarity in decentralized cooperative multi-agent deep reinforcement learning via multi-timescale learning," in *Conference on Lifelong Learning Agents*. PMLR, 2023, pp. 376–398.

[12] T. Rashid, M. Samvelyan, C. S. De Witt, G. Farquhar, J. Foerster, and S. Whiteson, "Monotonic value function factorisation for deep multi-agent reinforcement learning," *Journal of Machine Learning Research*, vol. 21, no. 178, pp. 1–51, 2020.

[13] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. Pieter Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[14] H. Hu and J. N. Foerster, "Simplified action decoder for deep multi-agent reinforcement learning," *arXiv preprint arXiv:1912.02288*, 2019.

[15] H. Hu, A. Lerer, B. Cui, L. Pineda, N. Brown, and J. Foerster, "Off-belief learning," in *International Conference on Machine Learning*. PMLR, 2021, pp. 4369–4379.

[16] B. Cui, H. Hu, L. Pineda, and J. Foerster, "K-level reasoning for zero-shot coordination in Hanabi," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 8215–8228.

[17] M. Zinkevich, M. Johanson, M. Bowling, and C. Piccione, "Regret minimization in games with incomplete information," in *Advances in Neural Information Processing Systems*, vol. 20, 2007.

[18] N. Brown, A. Lerer, S. Gross, and T. Sandholm, "Deep counterfactual regret minimization," in *International Conference on Machine Learning*. PMLR, 2019, pp. 793–802.

[19] J. Wang, Y. Li, S. Niu, and Z. Wu, "On deep CFR integrated with opponent model in imperfect information games," *Knowledge-Based Systems*, p. 114105, 2025.

[20] H. B. McMahan, G. J. Gordon, and A. Blum, "Planning in the presence of cost functions controlled by an adversary," in *Proceedings of the 20th International Conference on Machine Learning (ICML-03)*, 2003, pp. 536–543.

[21] M. Lanctot, V. Zambaldi, A. Gruslys, A. Lazaridou, K. Tuyls, J. Pérolat, D. Silver, and T. Graepel, "A unified game-theoretic approach to multiagent reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[22] R. Zhao, J. Song, Y. Hu, Y. Gao, Y. Wu, Z. Sun, and W. Yang, "Maximum entropy population-based training for zero-shot human-AI coordination," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.

[23] A. Lupu, B. Cui, H. Hu, and J. Foerster, "Trajectory diversity for zero-shot coordination," in *International Conference on Machine Learning*. PMLR, 2021.

[24] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.

[25] F. A. Oliehoek and C. Amato, *A Concise Introduction to Decentralized POMDPs*. Springer, 2016.

[26] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[27] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *International Conference on Learning Representations*, 2016.

[28] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *International Conference on Machine Learning*. PMLR, 2017, pp. 1126–1135.

Appendix A

Code Snippets

```
class RPPOActorCritic(nn.Module):
    hidden_dim:  int = HIDDEN_DIM   # 64
    num_actions: int = NUM_ACTIONS  # 8

    @nn.compact
    def __call__(self, obs: jnp.ndarray, h: jnp.ndarray):
        x      = nn.relu(nn.Dense(128, name="obs_proj")(obs))
        new_h, _ = nn.GRUCell(features=self.hidden_dim, name="gru")(h, x)
        trunk  = nn.relu(nn.Dense(128, name="trunk")(new_h))
        logits = nn.Dense(self.num_actions, name="actor")(trunk)
        value  = nn.Dense(1, name="critic")(trunk).squeeze(-1)
        # Partner head reads GRU carry (dim 64), not trunk (dim 128)
        partner_logits = nn.Dense(self.num_actions,
                                  name="partner_head")(new_h)
        return logits, value, new_h, partner_logits
```

Listing A.1: RPPOActorCritic forward pass. The partner-prediction head reads directly from the GRU carry `new_h`, not the wider trunk, so it captures temporal convention signals before the action-specific projection.

```
# --- Score blending (coupling) ---
if (partner_scores is not None
        and self._coupling_alpha > 0.0
        and len(partner_scores) > 0):
    n_overlap = min(N, len(partner_scores))
    p = np.array(partner_scores[:n_overlap], dtype=np.float64)
    scores[:n_overlap] = (
        (1.0 - self._coupling_alpha) * scores[:n_overlap]
        + self._coupling_alpha * p
    )

# --- Multiplicative-weights swap-regret update ---
```

```
T   = self._meta_game_count
eta = np.sqrt(np.log(N) / T)

G    = np.maximum(0.0, scores[np.newaxis, :] - scores[:, np.newaxis])
gain = G.sum(axis=0)
self._log_weights += eta * gain

log_w_shifted      = self._log_weights - self._log_weights.max()
w                  = np.exp(log_w_shifted)
self.meta_strategy = w / w.sum()

# Exploration floor gamma / N
floor = self._exploration_gamma / N
for _ in range(10):
    self.meta_strategy = np.maximum(self.meta_strategy, floor)
    self.meta_strategy /= self.meta_strategy.sum()
    if self.meta_strategy.min() >= floor - 1e-10:
        break
```

LISTING A.2: Meta-strategy update in `solve_meta_game()`. Scores are first blended with the partner agent's probe scores under coupling weight α, then fed to a swap-regret multiplicative-weights update with an exploration floor.

```
for iteration in range(1, args.xdo_iterations + 1):

    # 1. Collect shared episode batch with current joint policy
    key, k_joint = jax.random.split(key)
    shared_batch = run_joint_rppo_episodes(
        policy_a, policy_b, args.episodes_per_iter, key=k_joint
    )
    mean_score = float(np.mean([ep["score"] for ep in shared_batch]))

    # 2. Snapshot scores so each agent reads a stable partner view
    scores_a_snap = list(solver_a._profile_scores)
    scores_b_snap = list(solver_b._profile_scores)

    # 3. Train A || B concurrently (solve_meta_game + oracle + BC profile)
    if _parallel:
        with ThreadPoolExecutor(max_workers=2) as pool:
            fut_a = pool.submit(
                _train_rppo_agent,
                solver_a, shared_batch, scores_b_snap,
                args.ppo_updates, args.n_episodes_rppo,
                args.bc_rounds, device_a,
            )
            fut_b = pool.submit(
                _train_rppo_agent,
                solver_b, shared_batch, scores_a_snap,
```

```
                args.ppo_updates, args.n_episodes_rppo,
                args.bc_rounds, device_b,
            )
            policy_a, *_ = fut_a.result()
            policy_b, *_ = fut_b.result()
    else:
        policy_a, *_ = _train_rppo_agent(
            solver_a, shared_batch, scores_b_snap,
            args.ppo_updates, args.n_episodes_rppo,
            args.bc_rounds, device_a,
        )
        policy_b, *_ = _train_rppo_agent(
            solver_b, shared_batch, scores_a_snap,
            args.ppo_updates, args.n_episodes_rppo,
            args.bc_rounds, device_b,
        )

    # 4. Log divergence and outer-loop metrics
    l1 = float(np.sum(np.abs(
        solver_a.meta_strategy - solver_b.meta_strategy
    )))
    logger.log_metastrategy_divergence(iteration, l1)
```

LISTING A.3: XDO outer loop for the RPPO solver. Each iteration collects a shared episode batch with the current joint policy, snapshots both agents' profile scores for stable coupling, then trains agent A and agent B concurrently on separate devices via `ThreadPoolExecutor`.

```
# Separate GRU hidden states for BR model and BC profile
h_br_a = jnp.zeros(HIDDEN_DIM)
h_bc_a = jnp.zeros(HIDDEN_DIM)

while not done:
    cur = int(jnp.argmax(state.cur_player_idx))
    obs = jnp.array(_env_get_obs(state, prev_state, prev_act, cur))

    if cur == 0:   # Agent A's turn
        # Forward pass through frozen BR oracle
        br_logits, _, h_br_a, _ = _rppo_model.apply(
            {"params": ac_params_a["params"]}, obs, h_br_a
        )
        # Forward pass through selected BC profile (convention follower)
        bc_logits = _bc_net.apply({"params": bc_params_k}, obs, h_bc_a)

        # Interpolate: beta=0 -> pure BR, beta=1 -> pure BC
        final_logits  = (1.0 - beta_a) * br_logits + beta_a * bc_logits
        masked_logits = jnp.where(legal > 0, final_logits, -1e9)
        probs         = jax.nn.softmax(masked_logits)
```

```
action        = int(jax.random.choice(key, 8, p=probs))

# Advance BC hidden state with chosen action
h_bc_a = _bc_net.apply(
    {"params": bc_params_k}, h_bc_a,
    jnp.array(action, jnp.int32), method=_bc_net.gru_step,
)
```

LISTING A.4: Convention-conditioned belief (CB) blending at test time. Agent A maintains parallel GRU hidden states for its best-response (BR) model and the selected BC profile. At each of A's turns the action logits are interpolated: $(1-\beta)\,\ell_{\mathrm{BR}} + \beta\,\ell_{\mathrm{BC}}$, grounding the policy in the identified convention without discarding learned strategy.